

AI Impact on Job Sector Analysis using Python

Domain:

Human Resource(HR)

Objective:

1. **Analyze AI impact on jobs** - Understand whether AI is replacing or creating jobs
2. **Identify affected industries** - Find industries with high AI adoption
3. **Study automation risk** - Detect jobs with high/low risk
4. **Evaluate skill demand** - Identify important future skills
5. **Support decision-making** - Help students & professionals plan careers

Outcome:

- The project helps uncover key patterns, trends, and insights regarding the impact of Artificial Intelligence on the job market.
- The analysis provides a clear understanding of how AI adoption varies across different industries and job roles.
- It highlights which jobs are at higher risk of automation and which are evolving due to technological advancements.
- The study also identifies important skills such as Artificial Intelligence, Machine Learning, and Data Analysis that are increasingly required in the modern workforce.
- Overall, the findings support informed decision-making and help students and professionals prepare for future job opportunities in an AI-driven environment.

Dataset Information:

Source: Kaggle – AI Impact on Job Sector dataset by Sumeet Akash. Additionally used: Imported dataset file (ai_job_impact.xlsx) for analysis in Google Colab.

Year / Timeline: The dataset represents recent data related to Artificial Intelligence impact on job sectors. It was last updated on April 10, 2026 on Kaggle, and reflects current trends in AI adoption and workforce transformation.

Dataset Description:

Artificial Intelligence (AI) is rapidly transforming industries and reshaping the global workforce. This dataset provides a comprehensive view of how AI adoption affects jobs, salaries, productivity, and employment status across multiple sectors.

It contains 2,000 synthetic yet realistic records representing employees from diverse industries such as IT, Healthcare, Finance, Manufacturing, and more.

Dataset Size: Total Rows: 2000 & Total Columns: 17

Features: The dataset includes the following key attributes:

- * **Employee ID** – Unique identifier for each employee
- * **Age** – Age of the employee
- * **Gender** – Gender of the employee
- * **Education Level** – Qualification of the employee
- * **Industry** – Industry in which the employee works
- * **Job Role** – Specific job position
- * **Years of Experience** – Work experience in years
- * **AI Adoption Level** – Level of AI usage in the job

- * **Automation Risk** – Probability of job being automated
- * **Upskilling Required** – Indicates need for skill improvement
- * **Salary Before AI** – Salary before AI adoption
- * **Salary After AI** – Salary after AI adoption
- * **Job Status** – Status of job (retained, lost, etc.)
- * **Work Hours Per Week** – Number of working hours
- * **Remote Work** – Whether job is remote or not
- * **Job Satisfaction** – Employee satisfaction level
- * **Productivity Change %** – Change in productivity after AI

Types of Analysis:

Descriptive Analysis(summarizing the dataset):

Descriptive analysis is used to summarize and present the main features of the dataset. It involves analyzing data using statistical measures such as mean, median, and distribution to understand patterns and trends. In this project, descriptive analysis helps in identifying how AI adoption varies across industries, job roles, and experience levels.

Diagnostic Analysis (finding reasons behind patterns):

Diagnostic analysis is used to identify the reasons behind patterns and trends observed in the data. It focuses on understanding why certain outcomes occur by exploring relationships between variables.

In this project, diagnostic analysis helps explain why some jobs have higher automation risk, why salaries change after AI adoption, and how different factors influence job transformation.

Problem Definition:

The main goal of this project is to understand how Artificial Intelligence is impacting jobs across different industries. Even though we have the dataset, it doesn't directly tell us:

- which industries are most affected by AI,
- which job roles have higher automation risk,
- how AI adoption influences salary and productivity,
- whether jobs are being replaced, retained, or transformed,
- which skills are becoming more important due to AI,
- So, through this project, I want to explore the data properly and identify how AI is shaping the job market,
- which sectors show higher AI adoption,
- how job satisfaction and work patterns are changing, and
- what trends are present in workforce transformation.

In simple terms, the problem is to clearly understand how AI is affecting employment, skills, and job stability across different industries using the given dataset.

Expected Outcome:

After analyzing the dataset, I expect to gain a clear understanding of how Artificial Intelligence is influencing job sectors. By the end of the analysis, I should know:

- which industries and job roles are most impacted by AI,
- which jobs have higher or lower automation risk,
- how AI adoption affects salary and productivity,
- what skills are required for future job roles,
- how employment status changes due to AI,
- what patterns and trends exist in AI-driven job transformation, and
- what factors contribute to changes in job performance and satisfaction.

Overall, the expected outcome is to obtain meaningful insights that explain how AI is transforming the workforce and supporting better career planning and decision-making.

Import libraries & load dataset:

```
[ ] #Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

[ ] #Load Dataset
df = pd.read_excel('https://raw.githubusercontent.com/AhalyaAnbu/MyDataset/main/AI%20Job%20Impact.xlsx')
df
```

	Employee ID	Age	Gender	Education Level	Industry	Job Role	Years of Experience	AI Adoption Level	Automation Risk	Upskilling Required	Salary Before AI	Salary After AI	Job Status	Work Hours Per Week	Remote Work	Satisfaction
0	E0001	50	Female	Bachelor	Marketing	Content Creator	26	High	High	Yes	106820	95455	Replaced	45	No	
1	E0002	45	Male	High School	Manufacturing	Quality Inspector	19	Low	Low	Yes	74131	72013	Unchanged	36	Yes	

To begin the analysis, I first imported the essential Python libraries required for working with the dataset. These libraries help in performing various tasks such as data cleaning, preprocessing, analysis, and visualization.

The main libraries used in this project are:

- **Pandas** – for data manipulation and analysis
- **NumPy** – for numerical operations
- **Matplotlib** – for creating charts and visualizations
- **Seaborn** – for advanced statistical visualizations

After importing the libraries, I loaded the AI Job Impact dataset into a Pandas DataFrame using Python. This allowed the dataset to be viewed in a structured table format.

The dataset contains information related to:

- Employee demographics
- Industry and job role
- AI adoption level
- Automation risk
- Salary before and after AI
- Job status
- Productivity changes
- Work hours and remote work

The first few rows of the dataset were displayed to confirm that:

- The dataset loaded successfully
- All columns appeared correctly
- Data values were readable and properly structured

This initial step helped verify that the dataset was ready for further preprocessing and exploratory data analysis.

Dataset description (rows, columns, features):

```
[ ] ▶ #Rows
      Rows= df.shape[0]

      #Columns
      Columns= df.shape[1]

      #Features
      Features= df.columns

      print("Rows:", Rows)
      print("Columns:", Columns)
      print("Features:", Features)

▼ ... Rows: 2000
      Columns: 17
      Features: Index(['Employee ID', 'Age', 'Gender', 'Education Level', 'Industry',
                       'Job Role', 'Years of Experience', 'AI Adoption Level',
                       'Automation Risk', 'Upskilling Required', 'Salary Before AI',
                       'Salary After AI', 'Job Status', 'Work Hours Per Week', 'Remote Work',
                       'Job Satisfaction', 'Productivity Change %'],
                      dtype='object')
```

After loading the dataset, the overall structure of the data was examined to understand its size and features. The **df.shape** property was used to identify the total number of rows and columns present in the dataset.

The output confirmed that the dataset contains **2000 rows and 17 columns**, meaning there are 2000 employee records with 17 different attributes related to workforce and Artificial Intelligence impact analysis.

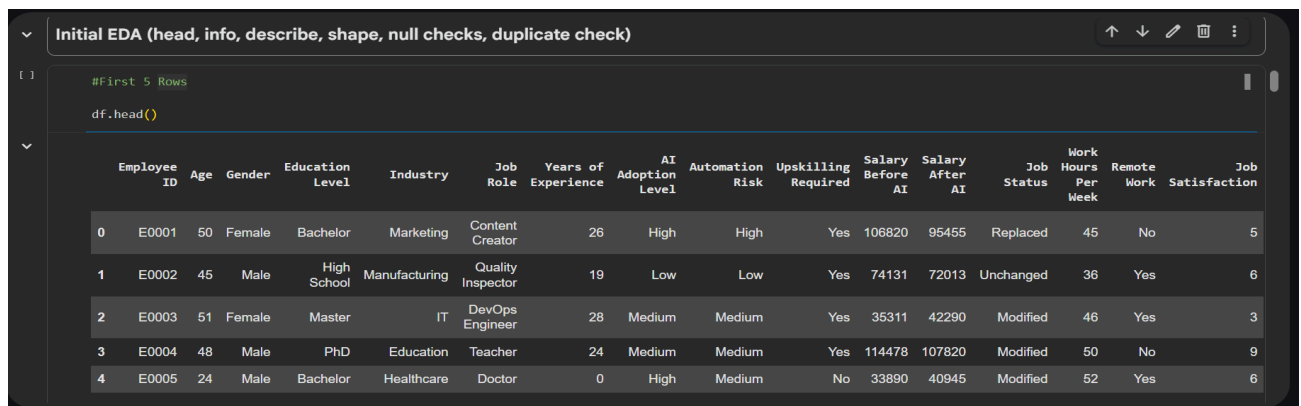
Next, the **df.columns** property was used to display all feature names available in the dataset.

These columns contain important information such as employee demographics, industry details, job roles, AI adoption levels, automation risk, salary information, productivity changes, remote work status, and employee satisfaction scores.

Initial EDA (head, info, describe, shape, null checks, duplicate check):

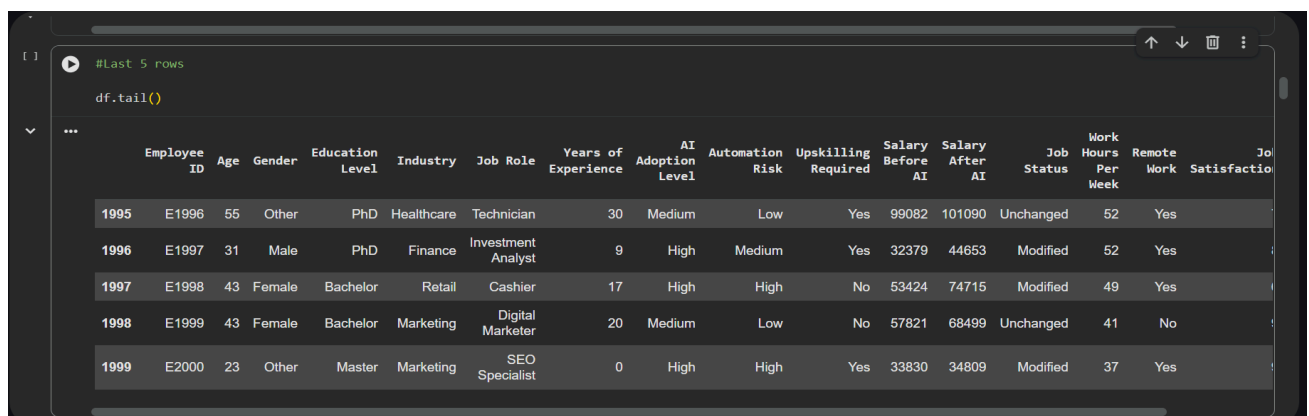
df.head(): This function is used to display the first 5 rows of the dataset. It provides a quick preview of the data and helps in understanding:

- Column names
- Data types
- Sample values
- Dataset structure



	Employee ID	Age	Gender	Education Level	Industry	Job Role	Years of Experience	AI Adoption Level	Automation Risk	Upskilling Required	Salary Before AI	Salary After AI	Job Status	Work Hours Per Week	Remote Work	Job Satisfaction
0	E0001	50	Female	Bachelor	Marketing	Content Creator	26	High	High	Yes	106820	95455	Replaced	45	No	5
1	E0002	45	Male	High School	Manufacturing	Quality Inspector	19	Low	Low	Yes	74131	72013	Unchanged	36	Yes	6
2	E0003	51	Female	Master	IT	DevOps Engineer	28	Medium	Medium	Yes	35311	42290	Modified	46	Yes	3
3	E0004	48	Male	PhD	Education	Teacher	24	Medium	Medium	Yes	114478	107820	Modified	50	No	9
4	E0005	24	Male	Bachelor	Healthcare	Doctor	0	High	Medium	No	33890	40945	Modified	52	Yes	6

df.tail(): This function is used to display the last 5 rows of the dataset. It helps to verify the ending records and understand how the dataset concludes.



	Employee ID	Age	Gender	Education Level	Industry	Job Role	Years of Experience	AI Adoption Level	Automation Risk	Upskilling Required	Salary Before AI	Salary After AI	Job Status	Work Hours Per Week	Remote Work	Job Satisfaction
1995	E1996	55	Other	PhD	Healthcare	Technician	30	Medium	Low	Yes	99082	101090	Unchanged	52	Yes	
1996	E1997	31	Male	PhD	Finance	Investment Analyst	9	High	Medium	Yes	32379	44653	Modified	52	Yes	
1997	E1998	43	Female	Bachelor	Retail	Cashier	17	High	High	No	53424	74715	Modified	49	Yes	
1998	E1999	43	Female	Bachelor	Marketing	Digital Marketer	20	Medium	Low	No	57821	68499	Unchanged	41	No	
1999	E2000	23	Other	Master	Marketing	SEO Specialist	0	High	High	Yes	33830	34809	Modified	37	Yes	

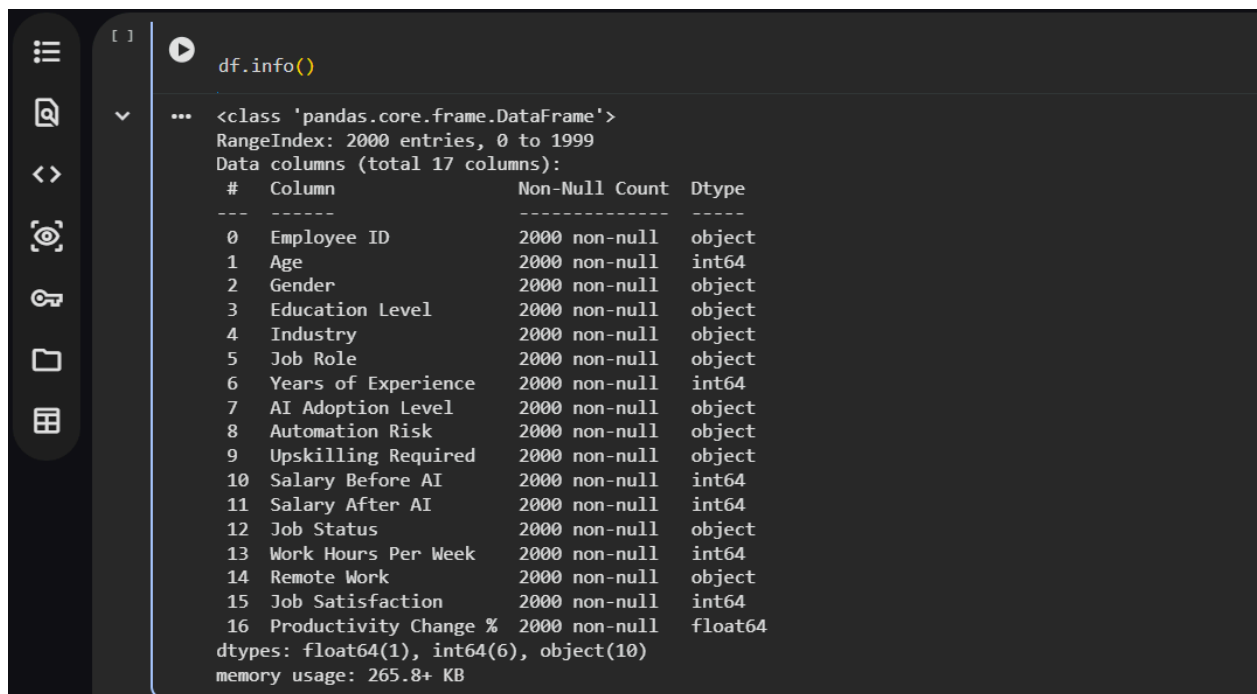
df.info(): This function is used to display detailed information about the dataset structure.

It provides a summary of the DataFrame including:

- Total number of rows and columns
- Column names
- Data types of each column
- Non-null value count
- Memory usage of the dataset

This function is important in Exploratory Data Analysis (EDA) because it helps identify:

- Missing values
- Categorical and numerical columns
- Data consistency
- Dataset size and structure



```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 17 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   Employee ID           2000 non-null  object 
 1   Age                   2000 non-null  int64  
 2   Gender                2000 non-null  object 
 3   Education Level       2000 non-null  object 
 4   Industry              2000 non-null  object 
 5   Job Role              2000 non-null  object 
 6   Years of Experience    2000 non-null  int64  
 7   AI Adoption Level     2000 non-null  object 
 8   Automation Risk       2000 non-null  object 
 9   Upskilling Required   2000 non-null  object 
10   Salary Before AI      2000 non-null  int64  
11   Salary After AI       2000 non-null  int64  
12   Job Status            2000 non-null  object 
13   Work Hours Per Week   2000 non-null  int64  
14   Remote Work           2000 non-null  object 
15   Job Satisfaction      2000 non-null  int64  
16   Productivity Change % 2000 non-null  float64
dtypes: float64(1), int64(6), object(10)
memory usage: 265.8+ KB
```

df.describe(): It generates descriptive statistics for numerical columns. It displays values such as count, mean, standard deviation, minimum, maximum, and quartiles & round (2) rounds all values to 2 decimal places for better readability.

[4] ✓ 0s # Summary Statistics

```
df.describe().round(2)
```

	Age	Years of Experience	Salary Before AI	Salary After AI	Work Hours Per Week	Job Satisfaction	Productivity Change %
count	2000.00	2000.00	2000.00	2000.00	2000.00	2000.00	2000.00
mean	40.56	16.66	73942.07	78428.64	44.85	6.02	9.79
std	10.79	10.75	26055.82	29351.60	5.71	2.01	17.19
min	22.00	0.00	30036.00	24447.00	35.00	3.00	-19.99
25%	32.00	8.00	51665.50	54086.75	40.00	4.00	-5.36
50%	40.00	16.00	74620.00	76820.50	45.00	6.00	9.85
75%	50.00	26.00	95418.25	100730.25	50.00	8.00	24.58
max	59.00	37.00	119976.00	161745.00	54.00	9.00	39.99

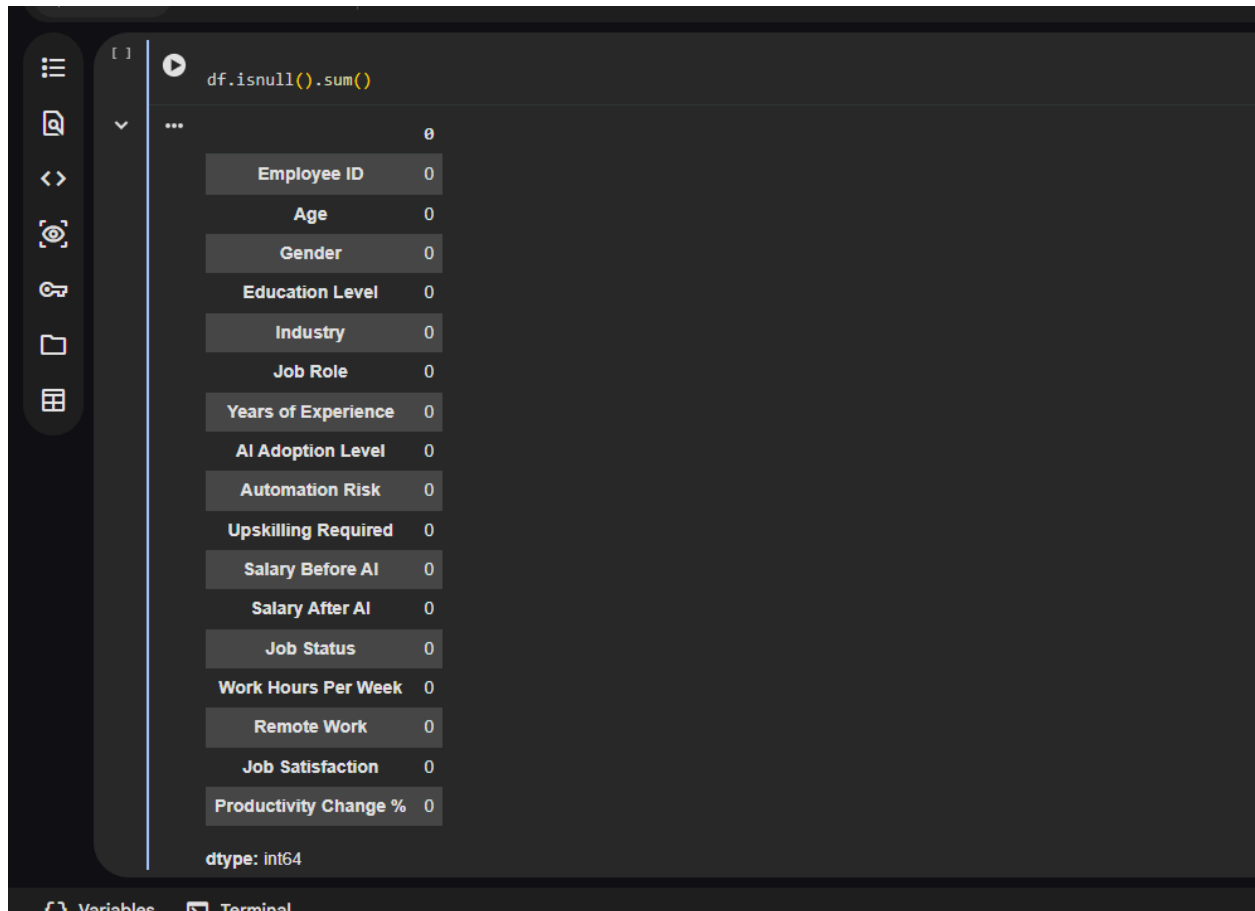
df.shape(): The following code is used to identify the total number of rows and columns present in the dataset.

- shape is a DataFrame property in Pandas.
- It returns the dimensions of the dataset in the format:

```
[ ] # Shape of dataset(rows & columns)
df.shape
... (2000, 17)
```

df.isnull(): Checks every cell in the dataset.,Returns True for missing/null values,Returns False for available values.

.sum():Counts the total number of True values in each column. Since True = 1 and False = 0, the sum gives the total count of missing values.

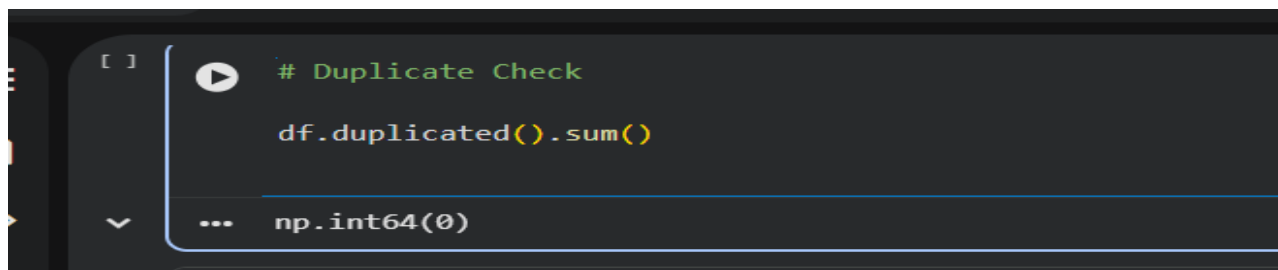


```
df.isnull().sum()
```

...	0
Employee ID	0
Age	0
Gender	0
Education Level	0
Industry	0
Job Role	0
Years of Experience	0
AI Adoption Level	0
Automation Risk	0
Upskilling Required	0
Salary Before AI	0
Salary After AI	0
Job Status	0
Work Hours Per Week	0
Remote Work	0
Job Satisfaction	0
Productivity Change %	0

dtype: int64

df.duplicated(): The following code is used to check whether the dataset contains any duplicate rows. Checks all rows in the dataset & Returns: True for duplicate rows, False for unique rows.



```
# Duplicate Check  
df.duplicated().sum()
```

```
... np.int64(0)
```

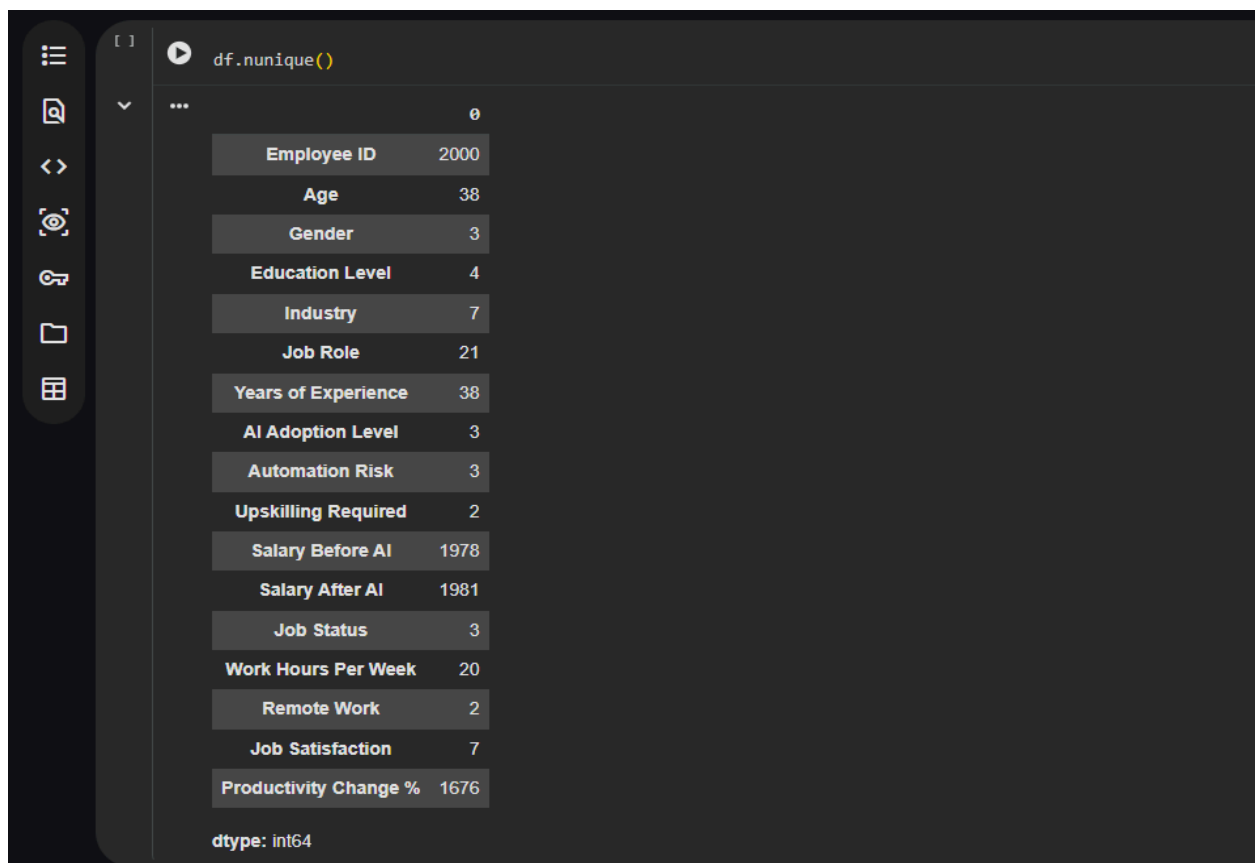
df.index: Index is a Pandas DataFrame property. It displays the row indexing information of the dataset. The output shows: Starting index ,Ending index ,Step size between rows

```
# Starting & ending index of the rows

df.index

RangeIndex(start=0, stop=2000, step=1)
```

df.nunique(): nunique() is a Pandas function used to count unique/distinct values in each column. It helps to understand: Data diversity, Categorical value distribution ,Identifier uniqueness ,Feature variability.



df.nunique()

...	0
Employee ID	2000
Age	38
Gender	3
Education Level	4
Industry	7
Job Role	21
Years of Experience	38
AI Adoption Level	3
Automation Risk	3
Upskilling Required	2
Salary Before AI	1978
Salary After AI	1981
Job Status	3
Work Hours Per Week	20
Remote Work	2
Job Satisfaction	7
Productivity Change %	1676

dtype: int64

describe(include='object'):

- Generates summary statistics for categorical (text/object) columns. Displays important information such as: Count , Unique values , Most frequent value , Frequency of the most common value.

```
[ ] # Summary of Categorical Columns
df.describe(include='object')
```

	Employee ID	Gender	Education Level	Industry	Job Role	AI Adoption Level	Automation Risk	Upskilling Required	Job Status	Remote Work
count	2000	2000	2000	2000	2000	2000	2000	2000	2000	2000
unique	2000	3	4	7	21	3	3	2	3	2
top	E1984	Male	Bachelor	Finance	Data Analyst	High	High	Yes	Unchanged	No
freq	1	709	529	300	112	705	682	1026	1093	1027

describe(include='number'):

- Generates statistical summaries only for numerical columns.
- It helps analyze the distribution and variation of numeric data.

```
[ ] # Summary of numeric columns
df.describe(include='number').round(2)
```

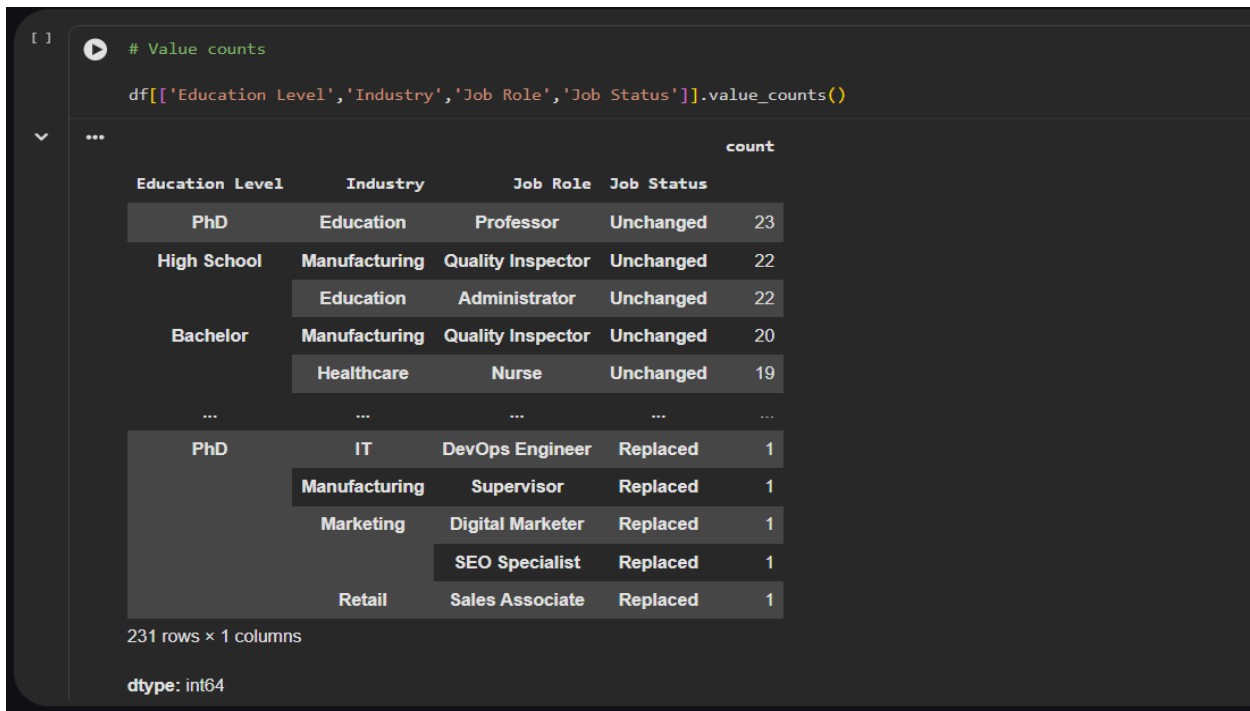
	Age	Years of Experience	Salary Before AI	Salary After AI	Work Hours Per Week	Job Satisfaction	Productivity Change %
count	2000.00	2000.00	2000.00	2000.00	2000.00	2000.00	2000.00
mean	40.56	16.66	73942.07	78428.64	44.85	6.02	9.79
std	10.79	10.75	26055.82	29351.60	5.71	2.01	17.19
min	22.00	0.00	30036.00	24447.00	35.00	3.00	-19.99
25%	32.00	8.00	51665.50	54086.75	40.00	4.00	-5.36
50%	40.00	16.00	74620.00	76820.50	45.00	6.00	9.85
75%	50.00	26.00	95418.25	100730.25	50.00	8.00	24.58
max	59.00	37.00	119976.00	161745.00	54.00	9.00	39.99

df[['Education Level','Industry','Job Role','Job Status']]:

- Selects the following categorical columns from the dataset: Education Level, Industry, Job Role, Job Status.

.value_counts():

- Counts the occurrence of each unique combination of values across the selected columns. Displays how many times each category combination appears in the dataset.



The screenshot shows a Jupyter Notebook interface with a code cell containing the following Python code:

```
# Value counts  
df[['Education Level','Industry','Job Role','Job Status']].value_counts()
```

Below the code cell, the output is displayed as a table with 5 columns: Education Level, Industry, Job Role, Job Status, and count. The table shows the frequency of each unique combination of these four categories. The output is truncated, showing only the first 10 rows.

Education Level	Industry	Job Role	Job Status	count
PhD	Education	Professor	Unchanged	23
High School	Manufacturing	Quality Inspector	Unchanged	22
	Education	Administrator	Unchanged	22
Bachelor	Manufacturing	Quality Inspector	Unchanged	20
	Healthcare	Nurse	Unchanged	19
...	...	...	...	...
PhD	IT	DevOps Engineer	Replaced	1
	Manufacturing	Supervisor	Replaced	1
	Marketing	Digital Marketer	Replaced	1
		SEO Specialist	Replaced	1
	Retail	Sales Associate	Replaced	1

Below the table, the dimensions and data type are shown:

231 rows x 1 columns
dtype: int64

df['Education Level']:

- Selects the **Education Level** column from the dataset.

.value_counts():

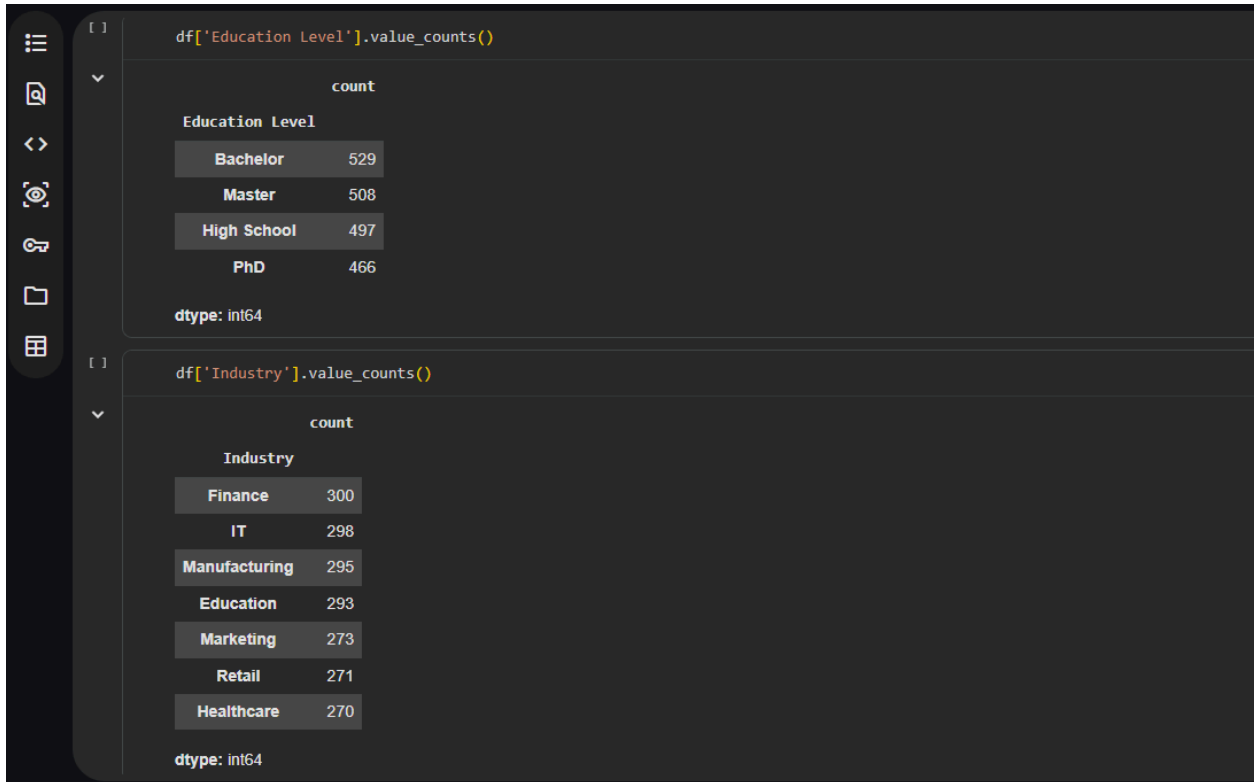
- Counts the frequency of each unique education category. Displays categories in descending order based on count.

df['Industry']:

- Selected the Industry column from the dataset.

.value_counts():

- Counts the number of employees in each industry category.



```
[ ] df['Education Level'].value_counts()

count
Education Level
Bachelor    529
Master      508
High School 497
PhD         466
dtype: int64

[ ] df['Industry'].value_counts()

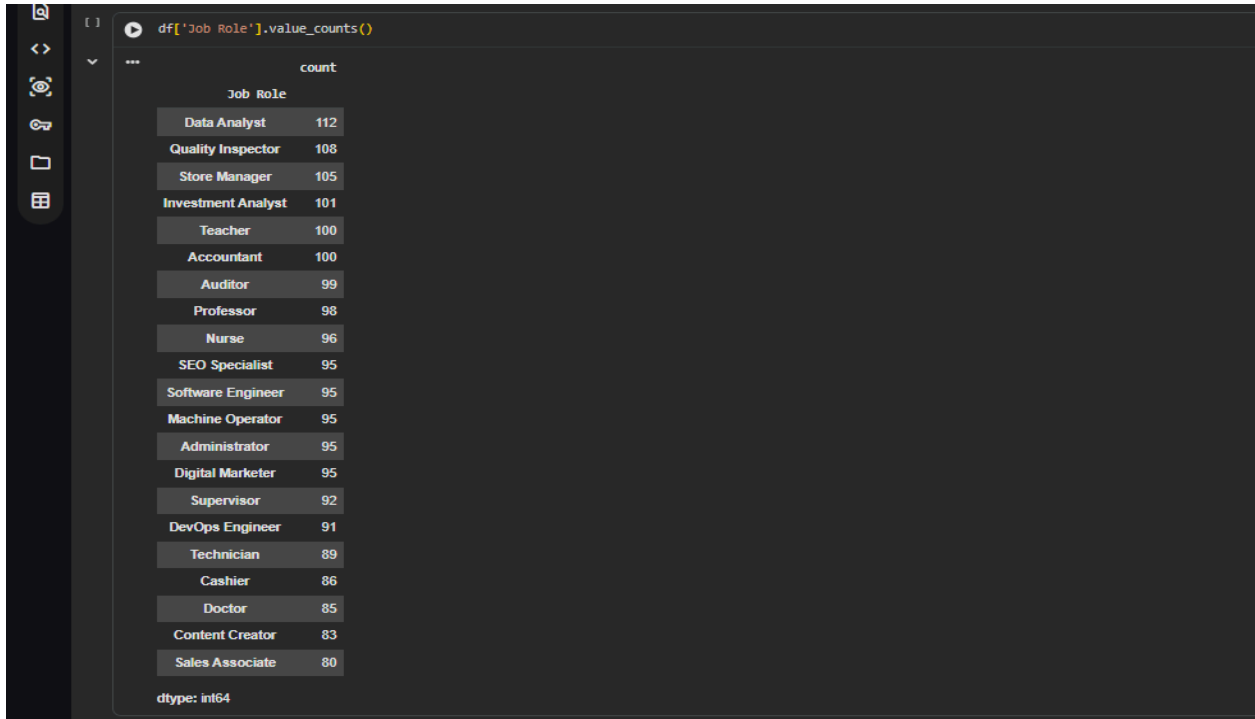
count
Industry
Finance    300
IT          298
Manufacturing 295
Education   293
Marketing   273
Retail      271
Healthcare  270
dtype: int64
```

df['Job Role']:

- Selects the **Job Role** column from the dataset.

.value_counts():

- Counts the number of occurrences of each unique job role.
- Displays the results in descending order based on frequency.



```
df['Job Role'].value_counts()
```

Job Role	count
Data Analyst	112
Quality Inspector	108
Store Manager	105
Investment Analyst	101
Teacher	100
Accountant	100
Auditor	99
Professor	98
Nurse	96
SEO Specialist	95
Software Engineer	95
Machine Operator	95
Administrator	95
Digital Marketer	95
Supervisor	92
DevOps Engineer	91
Technician	89
Cashier	86
Doctor	85
Content Creator	83
Sales Associate	80

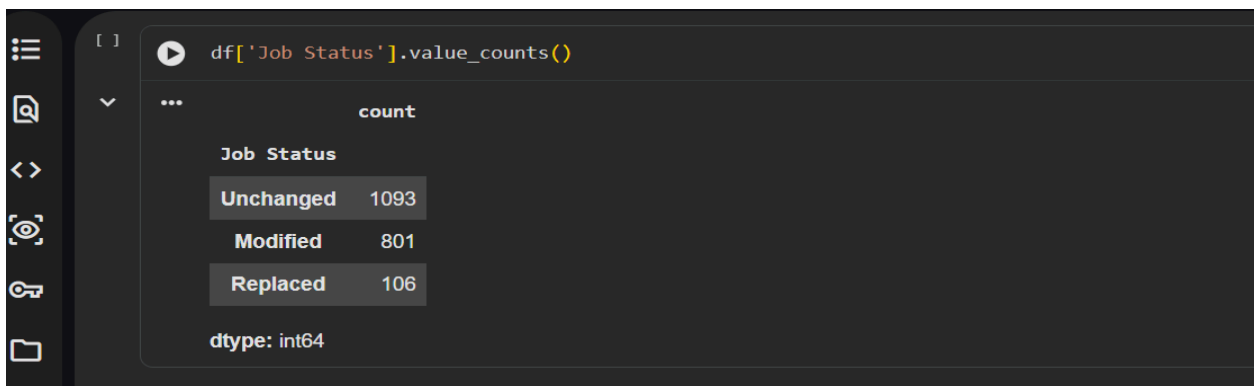
dtype: int64

df['Job Status']:

- Selects the **Job Status** column from the dataset.

.value_counts():

- Counts the number of occurrences of each unique job status category. Displays the categories in descending order based on frequency.



```
df['Job Status'].value_counts()
```

Job Status	count
Unchanged	1093
Modified	801
Replaced	106

dtype: int64

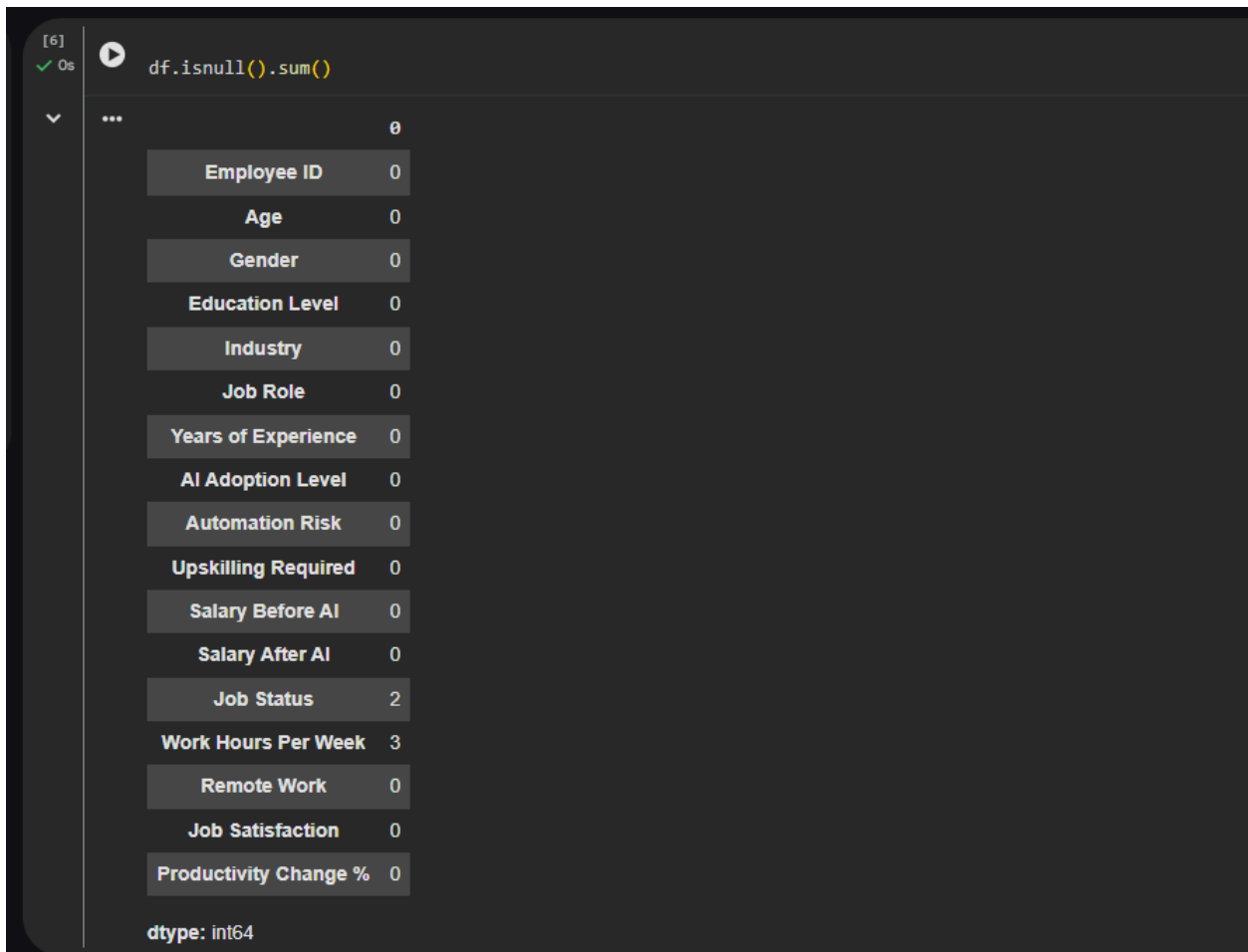
Data Cleaning & Pre-processing:

Handle Missing Values:

The `df.isnull().sum()` function is used to check for missing or null values in the dataset. Here, `isnull()` identifies whether any value is missing, and `sum()` counts the total number of missing values in each column.

The output shows the count of missing values for every column in the dataset.

- Most columns contain 0, which means there are no missing values.
- Job Status contains 2 missing values.
- Work Hours Per Week contains 3 missing values.



```
[6]
✓ Os df.isnull().sum()
```

...	0
Employee ID	0
Age	0
Gender	0
Education Level	0
Industry	0
Job Role	0
Years of Experience	0
AI Adoption Level	0
Automation Risk	0
Upskilling Required	0
Salary Before AI	0
Salary After AI	0
Job Status	2
Work Hours Per Week	3
Remote Work	0
Job Satisfaction	0
Productivity Change %	0

dtype: int64

Fill Numeric missing values with mean if dataset has null values:

The code is used to handle missing values in numerical columns of the dataset. First, `select_dtypes(include=['int64','float64'])` selects all numeric columns containing integer and float data types. The `.columns` function stores those column names in the variable `num_cols`. Then, a for loop iterates through each numerical column one by one.

The `fillna()` function replaces missing values in each column with the mean (average) value of that respective column using `df[col].mean()`. This technique helps maintain data consistency and prevents loss of important records during data preprocessing. It is commonly used in Exploratory Data Analysis (EDA) and machine learning to handle null values in numerical data.

```
# Fill numeric missing values with mean if dataset has null values
num_cols = df.select_dtypes(include=['int64','float64']).columns

for col in num_cols:
    df[col] = df[col].fillna(df[col].mean())
```

Fill Categorical missing values with mode if dataset has null values:

The code is used to handle missing values in categorical columns of the dataset. First, `select_dtypes(include='object')` selects all categorical columns containing text or object data types. The `.columns` function stores the column names in the variable `cat_cols`. Then, a for loop iterates through each categorical column one by one.

The `fillna()` function replaces missing values with the mode (most frequently occurring value) of that respective column using `df[col].mode()[0]`. This method helps preserve the most common category in the dataset and maintains data consistency.

Finally, the `print()` statement displays the message "Missing values handled successfully" indicating that the missing value handling process has been completed successfully.

```
[ ] ▶ # Fill categorical missing values with mode if dataset has null values
cat_cols = df.select_dtypes(include='object').columns

for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

print("Missing values handled successfully")

▼ ... Missing values handled successfully
```

Imputed:

The output shows 0 for all columns, which means the dataset does not contain any missing values. Both numerical and categorical missing values were successfully handled, indicating that the dataset is complete and clean.

```
[ ] ▶ # Handle Missing Values
df.isnull().sum()

▼ ...
Employee ID 0
Age 0
Gender 0
Education Level 0
Industry 0
Job Role 0
Years of Experience 0
AI Adoption Level 0
Automation Risk 0
Upskilling Required 0
Salary Before AI 0
Salary After AI 0
Job Status 0
Work Hours Per Week 0
Remote Work 0
Job Satisfaction 0
Productivity Change % 0

dtype: int64
```

Handle Duplicates-Check duplicate rows:

The code is used to check whether the dataset contains duplicate rows. The duplicated() function identifies rows that are repeated in the dataset, and the sum() function counts the total number of duplicate records. The output np.int64(0) indicates that there are no duplicate rows present in the dataset. This means every record is unique and no repeated employee data exists.

```
[ ] 1 # Handle Duplicates-Check duplicate rows:
    df.duplicated().sum()

    ... np.int64(0)
```

Remove Duplicates if dataset has duplicates:

The code is used to remove duplicate rows from the dataset if any duplicates are present. The `drop_duplicates()` function identifies repeated records and removes them from the DataFrame. The cleaned dataset is then reassigned back to the variable `df`.

```
1 # Remove duplicates if dataset has duplicates
  df = df.drop_duplicates()
```

Select numeric columns for check outliers:

The code is used to select all numerical columns from the dataset for outlier analysis. The `select_dtypes(include=['int64','float64'])` function identifies columns containing integer and float data types. These selected numerical column names are then stored in the variable `num_cols` using the `.columns` property.

```
# Select Numeric columns for Check Outliers
num_cols = df.select_dtypes(include=['int64','float64']).columns

num_cols

Index(['Age', 'Years of Experience', 'Salary Before AI', 'Salary After AI',
       'Work Hours Per Week', 'Job Satisfaction', 'Productivity Change %'],
      dtype='object')
```

Outliers checked with all numerical columns:

The code is used to detect and remove outliers from the numerical columns in the dataset using the **Interquartile Range (IQR) method**. First, important libraries such as `matplotlib` and `seaborn` are imported for data visualization. Then, a copy of the original dataset is created using `df.copy()` to avoid modifying the original data directly.

Next, the numerical columns such as Age, Years of Experience, Salary Before AI, Salary After AI, Work Hours Per Week, Job Satisfaction, and Productivity Change % are selected for outlier analysis. A for loop is used to process each numerical column one by one.

Inside the loop, the first quartile (Q1) and third quartile (Q3) are calculated using the `quantile()` function. The Interquartile Range (IQR) is then calculated using the formula:

$$IQR=Q3-Q1$$

Lower and upper boundaries are determined using:

$$Lower\ Bound=Q1-1.5\times IQR$$

$$Upper\ Bound=Q3+1.5\times IQR$$

Any values falling outside these boundaries are considered outliers and removed from the dataset.

Finally, boxplots are created using `seaborn. boxplot()` to compare the dataset before and after outlier treatment. The “Before Treating Outliers” plot shows the original data distribution, while the “After Treating Outliers” plot displays the cleaned data after removing extreme values.

This process helps improve data quality, reduce noise, and make the dataset more reliable for Exploratory Data Analysis (EDA) and machine learning models.

```
[1] # Outliers checked with All numerical columns
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Copy dataset
df_before = df.copy()
df_after = df.copy()
```

```
# Numeric columns from the dataset
num_cols = [
    'Age',
    'Years of Experience',
    'Salary Before AI',
    'Salary After AI',
    'Work Hours Per Week',
    'Job Satisfaction',
    'Productivity Change %'
]
```

```
# IQR Method - Remove Outliers
for col in num_cols:
    Q1 = df_after[col].quantile(0.25)
    Q3 = df_after[col].quantile(0.75)
    IQR = Q3 - Q1
```

```
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
```

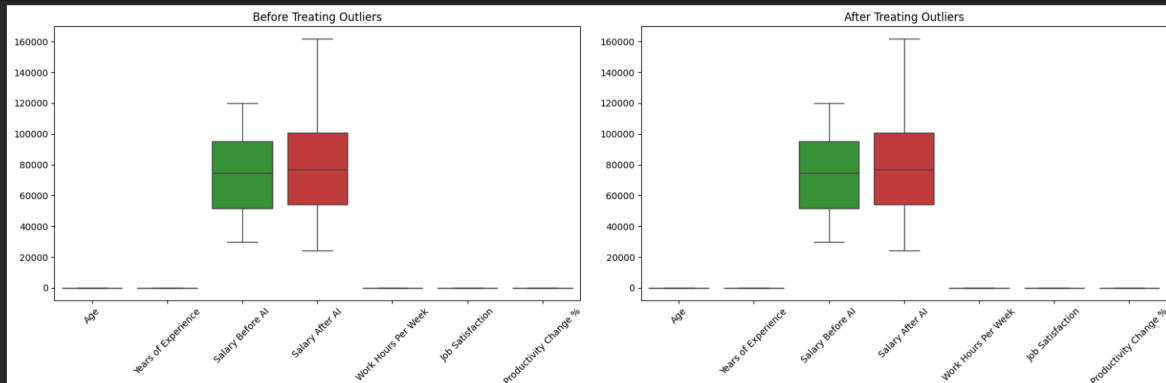
```
    df_after = df_after[
        (df_after[col] >= lower) &
        (df_after[col] <= upper)
    ]
```

```
# Plot Before & After
plt.figure(figsize=(18,6))
```

```
# Before
plt.subplot(1,2,1)
sns.boxplot(data=df_before[num_cols])
plt.title("Before Treating Outliers")
plt.xticks(rotation=45)
```

```
# After
plt.subplot(1,2,2)
sns.boxplot(data=df_after[num_cols])
plt.title("After Treating Outliers")
plt.xticks(rotation=45)
```

```
plt.tight_layout()
plt.show()
```



Outliers checked with the column of salary before AI & Salary After AI:

The code is used to detect and remove outliers specifically from the Salary Before AI and Salary After AI columns using the **Interquartile Range (IQR) method**. First, the matplotlib.pyplot and seaborn libraries are imported for data visualization. Then, copies of the dataset are created using df.copy() to preserve the original dataset while performing outlier treatment.

Finally, boxplots are generated using seaborn.boxplot() to compare the salary distributions before and after outlier treatment. The “Before Treating Outliers” graph displays the original salary distribution with extreme values, while the “After Treating Outliers” graph shows the cleaned distribution after removing outliers.

This process improves data quality and helps produce more accurate statistical analysis and machine learning results.

```
[ ]
24 # Outliers checked with the column of Salary Before AI & Salary After AI

import matplotlib.pyplot as plt
import seaborn as sns

# Copy dataset
df_before = df.copy()
df_after = df.copy()

# Numeric columns from the dataset
num_cols = [
    'Salary Before AI',
    'Salary After AI'
]

# IQR Method - Remove Outliers
for col in num_cols:
    Q1 = df_after[col].quantile(0.25)
    Q3 = df_after[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

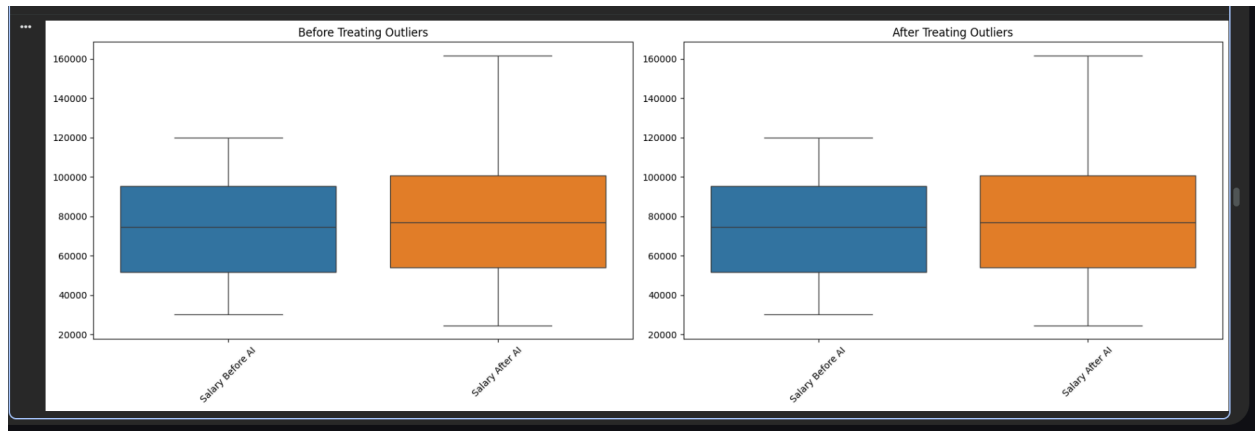
    df_after = df_after[
        (df_after[col] >= lower) &
        (df_after[col] <= upper)
    ]

# Plot Before & After
plt.figure(figsize=(18,6))

# Before
plt.subplot(1,2,1)
sns.boxplot(data=df_before[num_cols])
plt.title("Before Treating Outliers")
plt.xticks(rotation=45)

# After
plt.subplot(1,2,2)
sns.boxplot(data=df_after[num_cols])
plt.title("After Treating Outliers")
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```



Check Skewness & Apply transformation:

The code is used to check the skewness of numerical columns after data transformation. First, the `select_dtypes(include=['int64','float64'])` function selects all numerical columns from the dataset and stores them in the variable `numerical_cols`.

Then, the `skew()` function calculates the skewness value for each numerical feature. The `sort_values(ascending=False)` function arranges the skewness values in descending order, and `.round(2)` rounds the results to two decimal places for better readability.

Skewness measures the symmetry of data distribution. A skewness value close to 0 indicates that the data is normally distributed. Positive skewness means the data is slightly skewed to the right, while negative skewness indicates slight left skewness.

The output shows that most numerical columns have skewness values very close to zero:

- Salary After AI → 0.28
- Salary Before AI → 0.05
- Years of Experience → 0.04
- Productivity Change % → 0.03
- Job Satisfaction → 0.00
- Age → -0.03
- Work Hours Per Week → -0.07

These values indicate that the numerical features are nearly normally distributed after transformation and outlier treatment. This improves the quality of the dataset and makes it more suitable for statistical analysis, visualization, and machine learning models.

```
# Check Skewness & apply transformation
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns
skewness= df[numerical_cols].skew().sort_values(ascending=False).round(2)
print("Current skewness of numerical features after transformation:")
print(skewness)
```

```
Current skewness of numerical features after transformation:
Salary After AI      0.28
Salary Before AI     0.05
Years of Experience  0.04
Productivity Change % 0.03
Job Satisfaction     0.00
Age                 -0.03
Work Hours Per Week -0.07
dtype: float64
```

The code is used to visualize the skewness of numerical features in the dataset using a bar chart. First, the `matplotlib.pyplot` library is imported for data visualization. Then, the `select_dtypes(include=['int64','float64'])` function selects all numerical columns from the dataset and stores them in the variable `numerical_cols`.

Next, the `skew()` function calculates the skewness value of each numerical column, and `sort_values(ascending=False)` arranges the skewness values in descending order. Skewness helps identify whether the data distribution is symmetric or skewed toward the left or right side.

The `plt.figure(figsize=(10,6))` function creates the graph with a specified size. The `plt.bar()` function is used to create a bar chart where: X-axis represents numerical features & Y-axis represents skewness values

The chart title and axis labels are added using `plt.title()`, `plt.xlabel()`, `plt.ylabel()`, The `plt.axhline(0)` function draws a horizontal reference line at zero to easily compare positive and negative skewness values.

A for loop is then used to add skewness value labels above each bar using the `Plt. Text()` function. This improves readability and helps clearly identify the skewness value of every feature.

Finally, `plt.xticks(rotation=45)` rotates the feature names for better visibility, `plt.tight_layout()` adjusts spacing, and `plt.show()` displays the final graph. This visualization helps understand the distribution of numerical features and confirms whether data transformation and outlier treatment successfully reduced skewness in the dataset.

```
import matplotlib.pyplot as plt

# Select numerical columns
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns

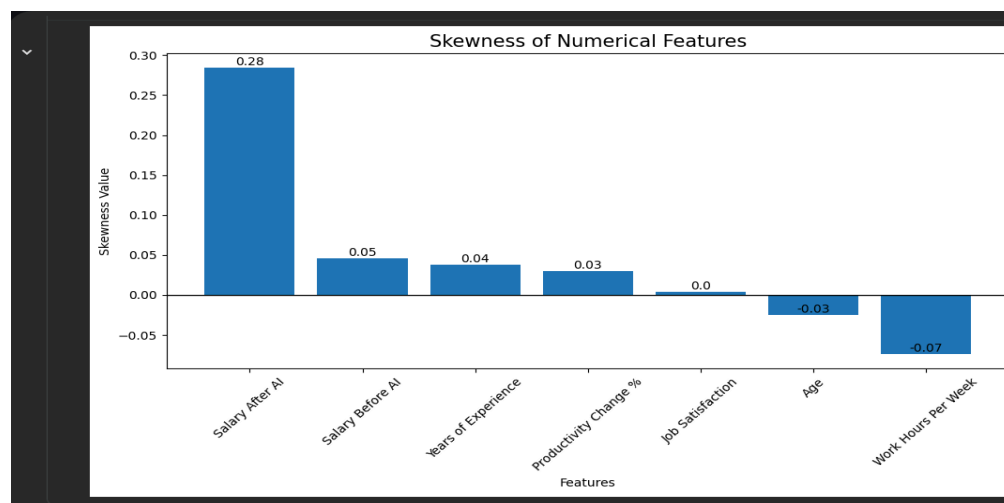
# Calculate skewness
skewness = df[numerical_cols].skew().sort_values(ascending=False)

# Plot
plt.figure(figsize=(10,6))
bars = plt.bar(skewness.index, skewness.values)

plt.title('Skewness of Numerical Features', fontsize=15)
plt.xlabel('Features')
plt.ylabel('Skewness Value')
plt.axhline(0, color='black', linewidth=1)

# Add value labels
for bar in bars:
    y = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, y,
             round(y, 2),
             ha='center',
             va='bottom')

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



The above graph represents the skewness values of numerical features in the dataset after data transformation and outlier treatment. Skewness measures how much the data distribution deviates from a normal distribution. A skewness value close to 0 indicates that the data is approximately symmetric and normally distributed. Positive values indicate right-skewed data, while negative values indicate left-skewed data.

From the graph, **Salary After AI** has the highest positive skewness value of **0.28**, indicating a slight right-skewed distribution. **Salary Before AI**, **Years of Experience**, and **Productivity Change %** also show very small positive skewness values, meaning their distributions are close to normal.

The **Job Satisfaction** feature has a skewness value of **0.0**, which indicates a perfectly balanced distribution. On the other hand, **Age** and **Work Hours Per Week** show slight negative skewness values of **-0.03** and **-0.07**, indicating a small left skew in the data distribution.

Overall, the graph confirms that all numerical features have skewness values very close to zero after preprocessing. This indicates that the dataset has a nearly normal distribution and is well-prepared for Exploratory Data Analysis (EDA), statistical analysis, visualization, and machine learning model building.

Convert Datatypes if needed:

The above code is used to convert categorical columns in the dataset into the category data type. First, the `select_dtypes(include='object')` function selects all columns containing object or text data types and stores their column names in the variable `cat_cols`.

Next, a for loop iterates through each categorical column one by one. The `astype('category')` function converts the data type of each selected column into the category type. This conversion helps optimize memory usage and improves processing efficiency during data analysis and machine learning tasks.

Finally, the `df.dtypes` statement displays the updated data types of all columns in the dataset.

```
[38] ✓ Os # Convert Datatypes if needed

cat_cols = df.select_dtypes(include='object').columns

for col in cat_cols:
    df[col] = df[col].astype('category')

df.dtypes
```

Employee ID	category
Age	int64
Gender	category
Education Level	category
Industry	category
Job Role	category
Years of Experience	int64
AI Adoption Level	category
Automation Risk	category
Upskilling Required	category
Salary Before AI	int64
Salary After AI	int64
Job Status	category
Work Hours Per Week	int64
Remote Work	category
Job Satisfaction	int64
Productivity Change %	float64

dtype: object

Feature Transformations : Derived Fields:

The above code is used to create new derived features related to salary changes after AI adoption. These new features help analyze how employee salaries were affected by AI implementation.

First, a new column named Salary Change is created by subtracting Salary Before AI from Salary After AI. This calculates the absolute increase or decrease in employee salary using the formula:

$$\text{Salary Change} = \text{Salary After AI} - \text{Salary Before AI}$$

Next, another column named Salary Change % is created to calculate the percentage change in salary after AI adoption. The percentage is calculated using the formula:

$$\text{Salary Change \%} = (\text{Salary After AI} - \text{Salary Before AI}) / \text{Salary Before AI} \times 100$$

The `.round(2)` function rounds the percentage values to two decimal places for better readability. The `print()` statement displays a message confirming that the new derived features have been successfully added.

Finally, the `display()` function shows the first few rows of the selected columns: Salary Before AI, Salary After AI, Salary Change, Salary Change %.

The output indicates both positive and negative salary changes. Positive values represent salary increases after AI adoption, while negative values indicate salary reductions. These derived features provide deeper insights into the financial impact of AI on employees and support further Exploratory Data Analysis (EDA), visualization, and predictive modeling.

```
Feature Transformations: Derived Fields

Created new features to analyze the impact of AI on salaries more effectively. Specifically,calculated the absolute change in salary and the percentage change in salary after AI adoption.

[39] df['Salary Change'] = df['Salary After AI'] - df['Salary Before AI']
✓ 0s df['Salary Change %'] = ((df['Salary After AI'] - df['Salary Before AI']) / df['Salary Before AI']) * 100.
df['Salary Change %'] = df['Salary Change %'].round(2)

print("New derived features added: 'Salary Change' and 'Salary Change %'.")
display(df[['Salary Before AI', 'Salary After AI', 'Salary Change', 'Salary Change %']].head())
```

... New derived features added: 'Salary Change' and 'Salary Change %'.

	Salary Before AI	Salary After AI	Salary Change	Salary Change %
0	106820	95455	-11365	-10.64
1	74131	72013	-2118	-2.86
2	35311	42290	6979	19.76
3	114478	107820	-6658	-5.82
4	33890	40945	7055	20.82

Cleaned Dataset After Transformation:

The above code is used to export the cleaned and transformed dataset into an Excel file. The `to_excel()` function saves the DataFrame `df` as an Excel file with the name `cleaned_dataset.xlsx`.

```
# Cleaned Dataset after Transformations
df . to_excel('cleaned_dataset.xlsx')
```

Data Visualization:

Univariate Analysis-Distribution of single variables(countplot,histogram,boxplot):

1.Countplot:

Code:

```
# Countplot

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6,4))
sns.countplot(x='AI Adoption Level', data=df,color='cyan',alpha=0.7)
plt.title('Distribution of AI Adoption Level')
plt.xlabel('AI Adoption Level')
plt.ylabel('Count')
plt.show()
```

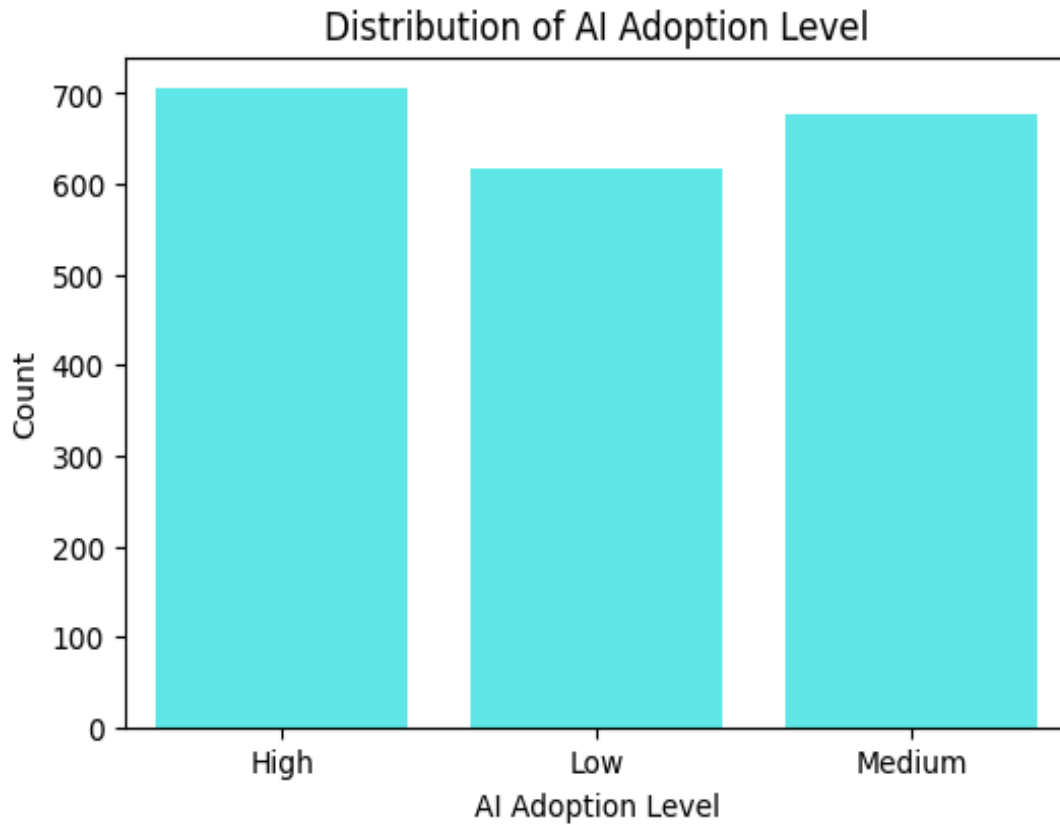
The above code is used to create a **countplot** for the **AI Adoption Level** column using the Seaborn visualization library. First, the matplotlib.pyplot and seaborn libraries are imported for creating graphs and visualizations.

The plt.figure(figsize=(6,4)) function creates a figure with a width of 6 inches and height of 4 inches.

Then, the sns.countplot() function is used to display the frequency count of each category in the AI Adoption Level column from the dataset df. The parameter color='cyan' sets the bar color to cyan, and alpha=0.7 controls the transparency of the bars.

The plt.title() function adds the chart title as "**Distribution of AI Adoption Level**". The plt.xlabel() and plt.ylabel() functions label the X-axis as **AI Adoption Level** and the Y-axis as **Count**. Finally, plt.show() displays the completed visualization.

Output:



Interpretation of Count plot:

Count plot- Shows the count/frequency of each category in a single categorical variable.

- This chart shows employee distribution across AI adoption levels.
- Feature-AI Adoption Level
- It helps understand whether AI adoption is Low, Medium, or High.
- From this, we identify the overall AI adoption pattern.

2.Histogram:

Code:

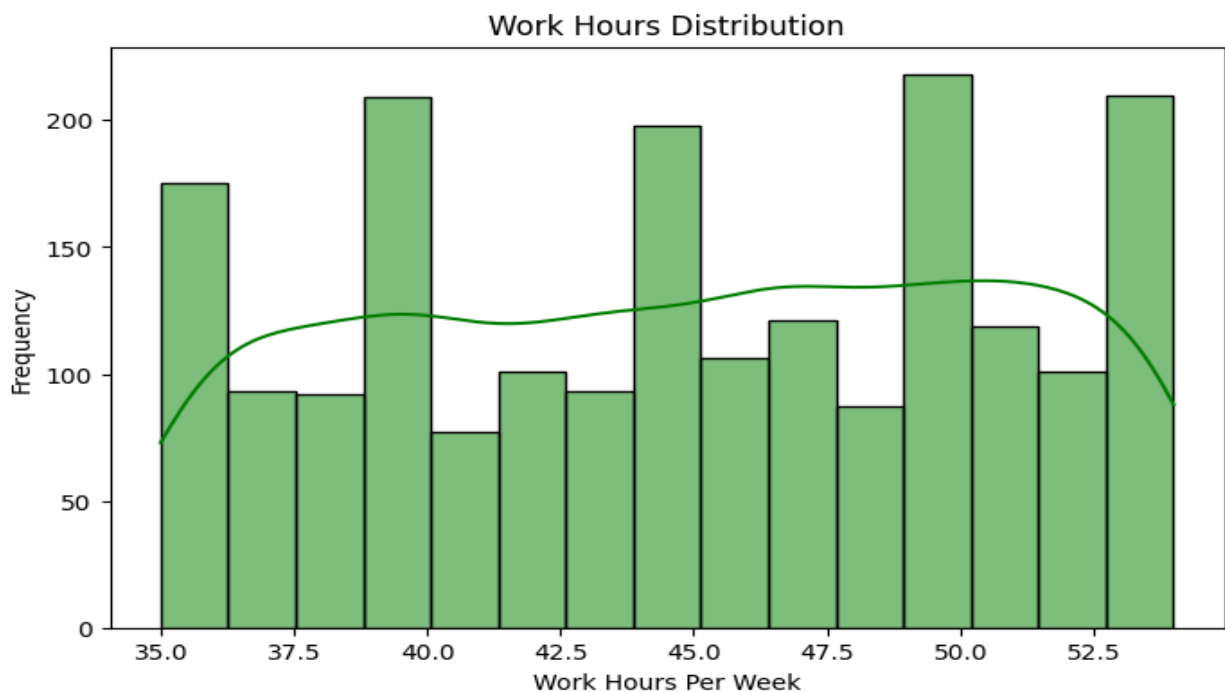
```
# Histogram

plt.figure(figsize=(8,5))
sns.histplot(df['Work Hours Per Week'], bins=15, kde=True,color='green')
plt.title('Work Hours Distribution')
plt.xlabel('Work Hours Per Week')
plt.ylabel('Frequency')
plt.show()
```

The `sns.histplot()` function from the Seaborn library is used to generate the histogram for the Work Hours Per Week column.

The parameter `bins=15` divides the data into 15 intervals or groups for better distribution analysis. The `kde=True` parameter adds a smooth density curve on top of the histogram, helping visualize the overall data distribution pattern. The `color='green'` parameter sets the histogram color to green.

Output:



Interpretation of Histogram:

Histogram- Shows the distribution of a single numerical variable using bars.

- This chart shows working hours distribution.
- Feature-Work Hours Per Week
- Histogram helps understand average workload patterns.
- From this, we identify common weekly work-hour ranges.

3.Boxplot:

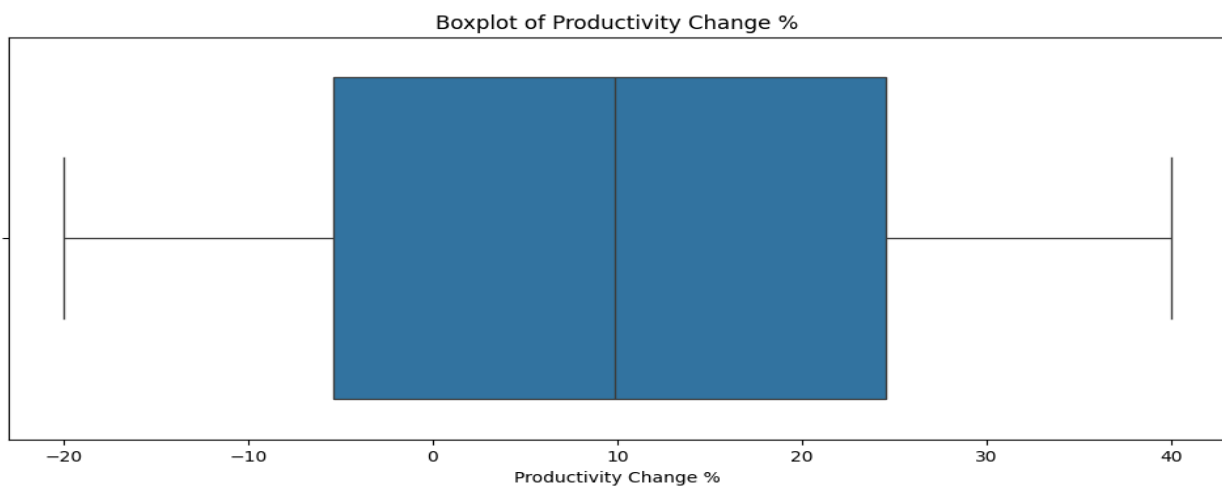
Code:

```
# Boxplot

plt.figure(figsize=(12,5))
sns.boxplot(x=df['Productivity Change %'],vert=False)
plt.title('Boxplot of Productivity Change %')
plt.xlabel('Productivity Change %')
plt.show()
```

The sns.boxplot() function is used to generate the boxplot for the Productivity Change % column.

Output:



Interpretation of Boxplot:

Boxplot- Shows data spread, median, quartiles, and outliers in a numerical variable.

- This chart shows productivity changes after AI adoption.
- Feature- Productivity Change %
- Boxplot helps identify spread, median, and unusual extreme changes.
- From this, we understand how AI impacts productivity.

Bivariate Analysis → relation between two variables (scatterplot, barplot, correlation heatmap):

4.Scatterplot:

Code:

```
# Scatter plot

plt.figure(figsize=(10,6))

plt.scatter(
    df['Salary Before AI'],
    df['Salary After AI'],
    color='orange',
    edgecolors='black',
    alpha=0.6,
    s=60
)

plt.title('Salary Before AI vs Salary After AI')
plt.xlabel('Salary Before AI')
plt.ylabel('Salary After AI')
plt.grid(alpha=0.3)

plt.show()
```

The `plt.scatter()` function plots the salary values where Salary Before AI is placed on the X-axis and Salary After AI is placed on the Y-axis. The parameter `color='orange'` changes the point color to orange, and `edgecolors='black'` adds black borders around the points.

The $\alpha=0.6$ parameter controls the transparency of the points, while $s=60$ sets the point size.

Output:



Interpretation of Scatter plot:

Scatterplot- Shows the relationship between two numerical variables.

- In this chart Shows relationship between Salary Before AI & Salary After AI
- Helps identify salary trend.
- Used features: Salary Before AI, Salary After AI

5.Bar Plot:

Code:

```
# Bar Plot

avg_productivity = df.groupby('AI Adoption Level')['Productivity Change %'].mean()

plt.figure(figsize=(7,5))
plt.bar(avg_productivity.index, avg_productivity.values,linestyle="--",color='purple',edgecolor='black',alpha=0.8)

plt.title('AI Adoption Level vs Productivity Change %')
plt.xlabel('AI Adoption Level')
plt.ylabel('Average Productivity Change %')

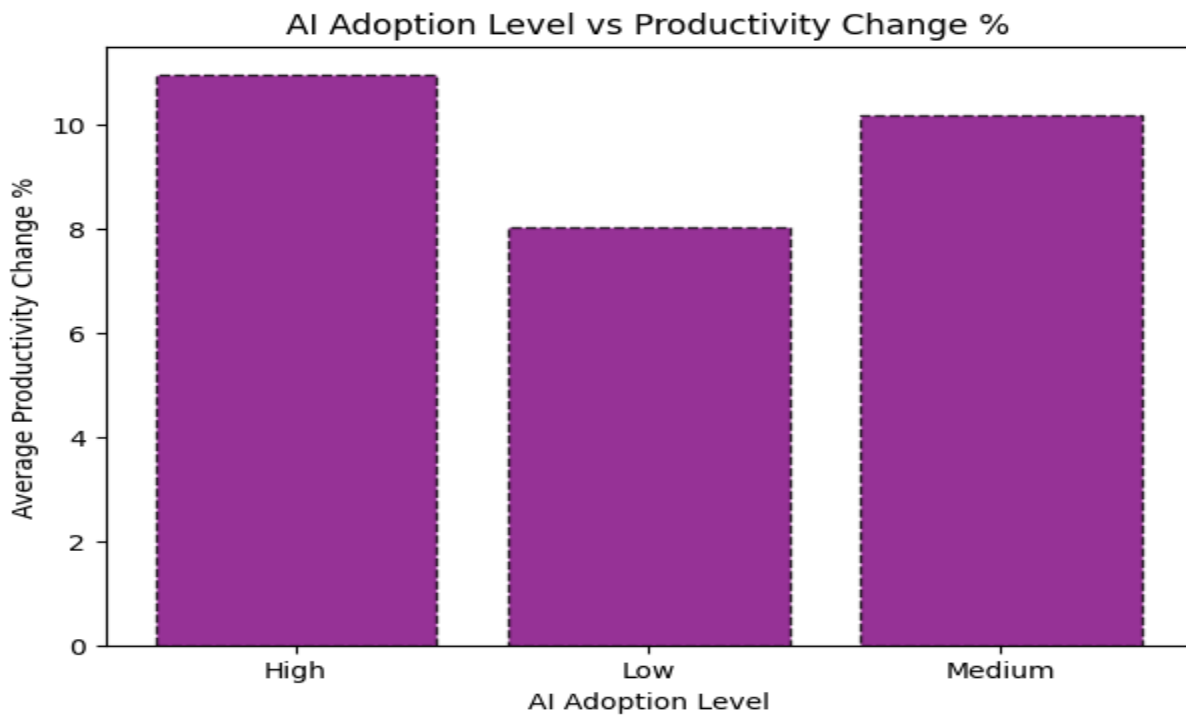
plt.show()
```

The plt.bar() function is used to create the bar chart where:

- avg_productivity.index represents the AI Adoption Level categories on the X-axis
- avg_productivity.values represents the average productivity change values on the Y-axis

The parameter color='purple' changes the bar color to purple, edgecolor='black' adds black borders to the bars, and alpha=0.8 controls the transparency of the bars.

Output:



Interpretation of Bar plot:

Barplot- Compares values between different categories.

- In this chart Compares productivity across AI adoption levels.
- Helps identify whether higher AI adoption improves productivity.
- Features used: AI Adoption Level, Productivity Change %.

6.Correlation Heatmap:

Code:

```
# Correlation Heatmap

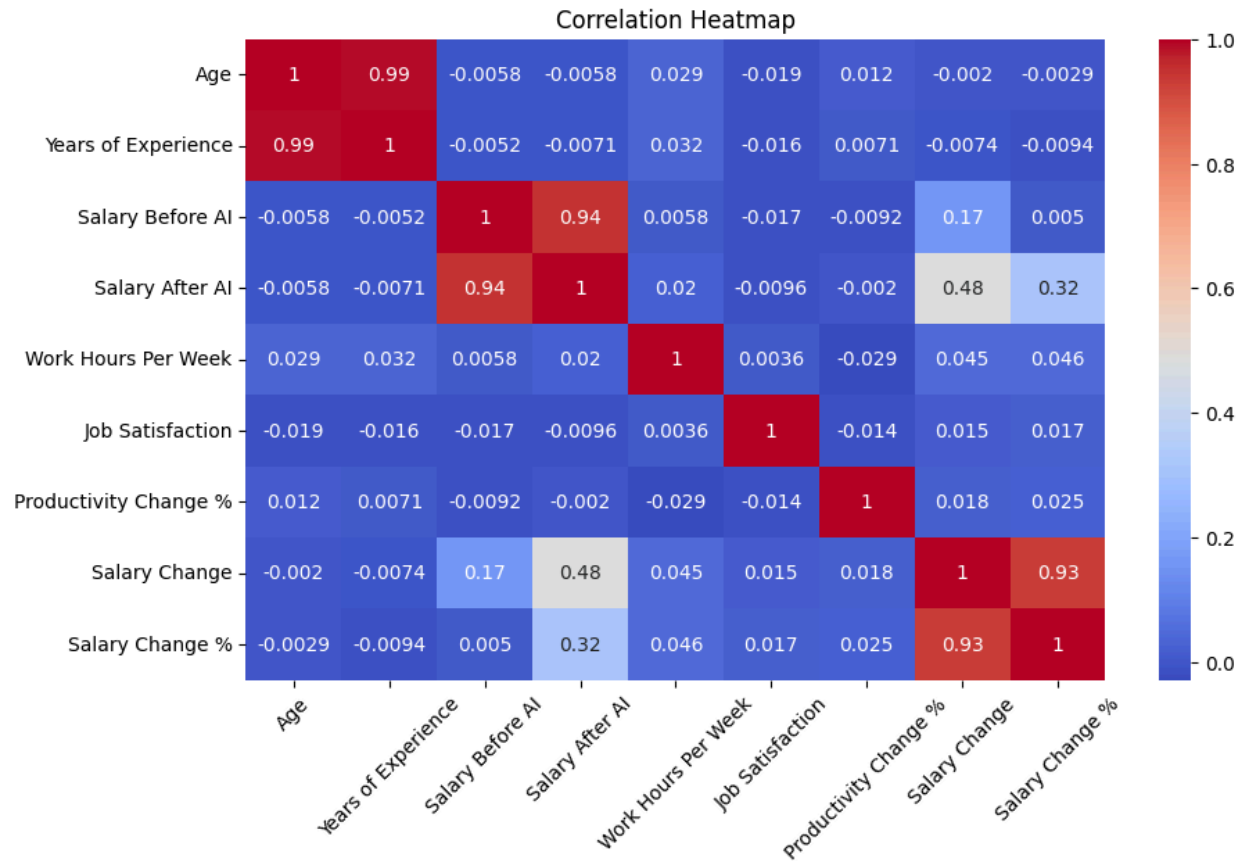
plt.figure(figsize=(10,6))
numeric_df = df.select_dtypes(include=['int64','float64'])
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.xticks(rotation=45)
plt.show()
```

The `select_dtypes(include=['int64','float64'])` function selects all numerical columns from the dataset and stores them in the variable `numeric_df`.

The `corr()` function calculates the correlation values between the numerical features.

The `sns.heatmap()` function creates the heatmap using the correlation values. The parameter `annot=True` displays the correlation numbers inside the boxes, and `cmap='coolwarm'` adds color variations to represent positive and negative correlations.

Output:



Interpretation of Correlation Heatmap:

Correlation Heatmap- Shows correlation strength between numerical variables using colors.

- This heatmap shows correlation between numerical variables.
- Features- Used All numeric features
- Dark/high values indicate strong positive relationship & Negative values indicate inverse relationship.
- From this, we identify which features strongly influence each other.

Multivariate Analysis → relation among 3+ variables (pairplot, grouped analysis, pivot tables, advanced plots):

7.Pairplot:

Code:

```
# Pair Plot

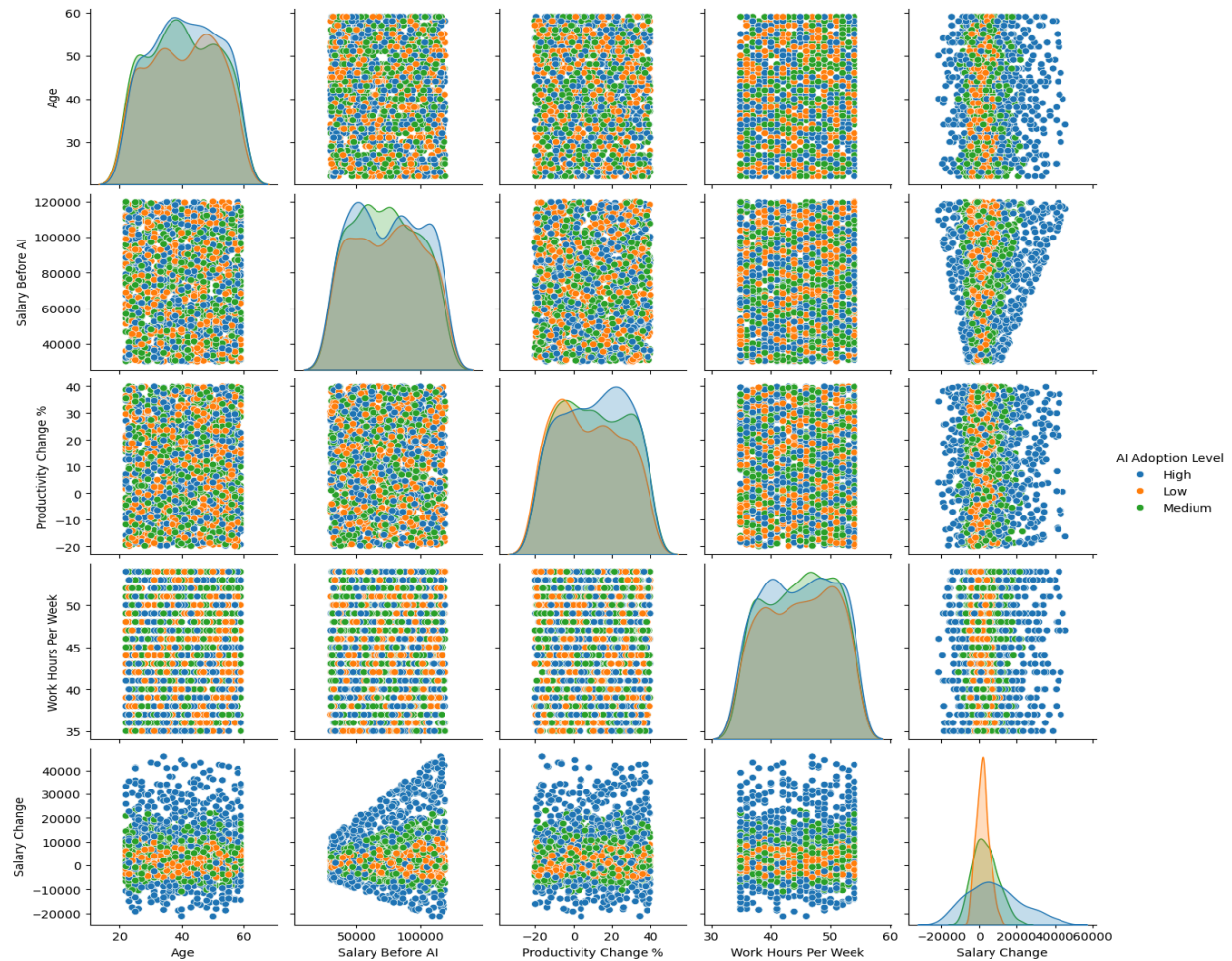
cols = [
    'Age',
    'Salary Before AI',
    'Productivity Change %',
    'Work Hours Per Week',
    'Salary Change'
]
sns.pairplot(
    df[cols + ['AI Adoption Level']],
    hue='AI Adoption Level'
)
plt.show()
```

The `sns.pairplot()` function creates pairwise graphs between these numerical columns.

Selected the chosen columns along with the AI Adoption Level column from the dataset.

Adds different colors based on the AI Adoption Level categories. This helps compare the data distribution and relationships between different AI adoption groups.

Output:



Interpretation of Pair Plot:

Pairplot- Shows pairwise relationships between multiple numerical variables at once.

- Features used: Age, Salary Before AI, Productivity Change %, Work Hours Per Week, Salary Change
- AI Adoption Level is used as color grouping.
- It helps identify Which variables move together, Strong/weak relationships, Group patterns by AI adoption, Outliers

8.Grouped Analysis:

Code:

```
# Grouped Analysis

grouped = df.groupby(
    ['Industry', 'AI Adoption Level']
)['Productivity Change %'].mean().unstack().round(2)

print(grouped)
```

AI Adoption Level	High	Low	Medium
Industry			
Education	10.91	6.35	11.87
Finance	12.93	7.64	7.93
Healthcare	8.99	7.39	11.30
IT	11.21	6.29	10.00
Manufacturing	12.39	9.97	9.92
Marketing	10.79	9.47	10.60
Retail	9.75	9.26	9.49

The `groupby()` function groups the dataset using the Industry and AI Adoption Level columns. The `mean()` function calculates the average value of Productivity Change % for each group.

The `unstack()` function converts the grouped data into a table format where AI Adoption Levels appear as separate columns. The `.round(2)` function rounds the values to two decimal places for better readability. Finally, the `print(grouped)` statement displays the grouped analysis table.

Interpretation of Grouped Analysis:

Grouped Analysis- Compares one variable across multiple grouped categories.

- It helps to identify which industries gain more productivity from AI.
- Features: Compares Industry + AI Adoption Level + Productivity Change %
- Shows impact of AI adoption across sectors.

9.Pivot Table:

Code:

```
# Pivot Table

pivot = pd.pivot_table(
    df,
    values='Salary After AI',
    index='Education Level',
    columns='Job Status',
    aggfunc='mean'
).round(2)

print(pivot)
```

...	Job Status	Modified	Replaced	Unchanged
	Education Level			
	Bachelor	78191.01	86424.10	79545.74
	High School	83356.56	87076.41	76348.47
	Master	77639.13	86758.94	78956.77
	PhD	76933.47	80838.33	74428.89

The `pd.pivot_table()` function creates a summarized table from the dataset.

The `values='Salary After AI'` parameter selects the Salary After AI column for calculation. The `index='Education Level'` parameter places Education Level as the rows of the table.

The `columns='Job Status'` parameter creates separate columns for different job status categories such as Modified, Replaced, and Unchanged.

The `aggfunc='mean'` parameter calculates the average salary for each category combination. The `.round(2)` function rounds the values to two decimal places for better readability. Finally, the `print(pivot)` statement displays the pivot table.

Interpretation of Pivot Table:

Pivot Table- Summarizes data in table format for multi-variable comparison.

- It Helps to understand salary changes by qualification and employment status.
- Features- Compares Education Level + Job Status + Salary After AI
- Shows which education groups benefit more.

9.Multi Category Analysis with pivot table:

Code:

```
# Multi Category Analysis with Pivot table

table = pd.pivot_table(
    df,
    values='Job Satisfaction',
    index='Gender',
    columns='Remote Work',
    aggfunc='mean'
)

print(table)
```

Remote Work	No	Yes
Gender		
Female	6.014493	5.908795
Male	6.025000	6.020057
Other	5.990683	6.160883

The `pd.pivot_table()` function creates a summarized table from the dataset.

The `values='Job Satisfaction'` parameter selects the Job Satisfaction column for calculation. The `index='Gender'` parameter places Gender as the rows of the table. The `columns='Remote Work'` parameter creates separate columns for employees who work remotely and those who do not.

The `aggfunc='mean'` parameter calculates the average job satisfaction score for each group. Finally, the `print(table)` statement displays the pivot table.

Interpretation of Multi Category Analysis:

- It helps to understand satisfaction differences among employee groups.
- Features: Compares Gender + Remote Work + Job Satisfaction
- Shows impact of remote work on satisfaction

10. Advanced Plot- Bubble Chart:

Code:

```
# Advanced Plot- Bubble Chart:

# Size mapping for Automation Risk
size_map = {
    'Low': 100,
    'Medium': 250,
    'High': 400
}

sizes = df['Automation Risk'].map(size_map)
plt.figure(figsize=(10,6))
scatter = plt.scatter(
    df['Years of Experience'],
    df['Salary After AI'],
    s=sizes,
    c=df['Job Satisfaction'],
    alpha=0.6
)

plt.colorbar(scatter, label='Job Satisfaction')
plt.title('Experience vs Salary After AI (Bubble = Automation Risk)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary After AI')
plt.show()
```

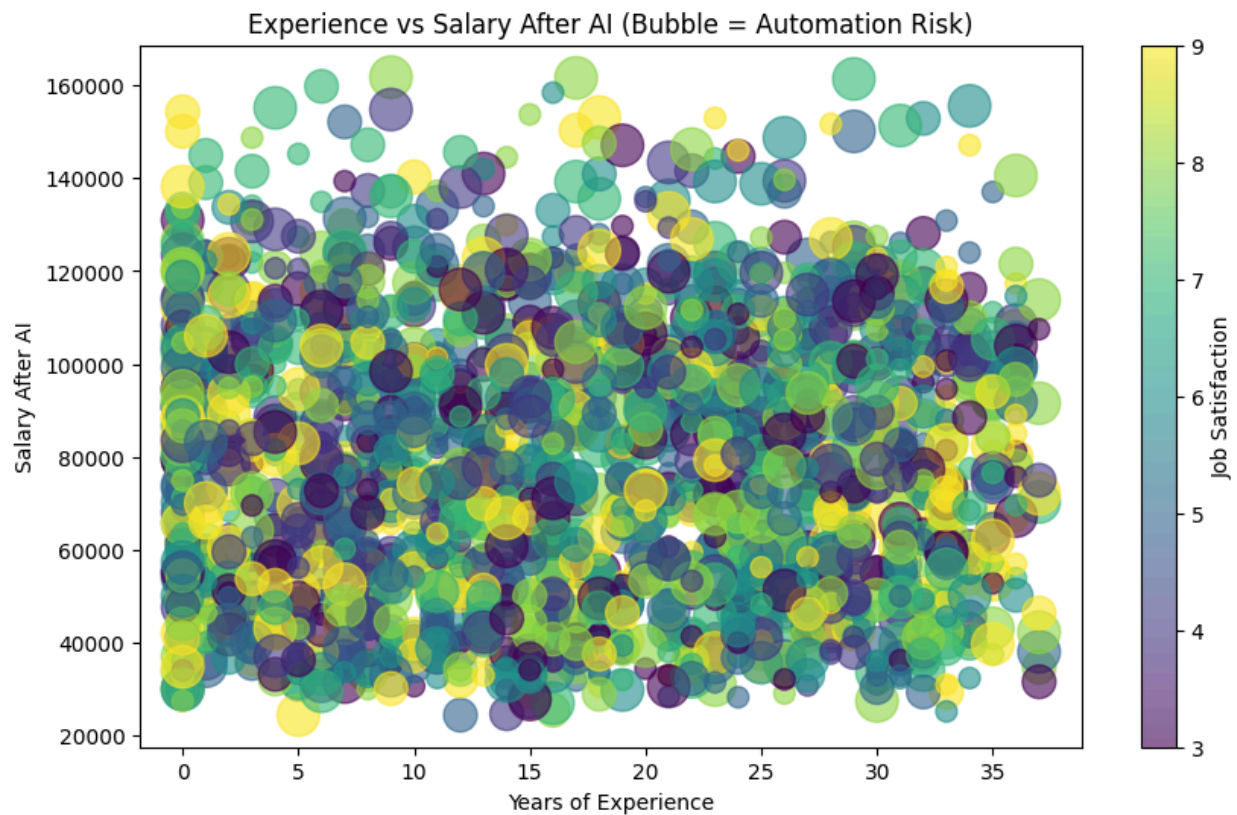
The `map()` function converts the Automation Risk categories into bubble sizes and stores them in the variable `sizes`. The `plt.figure(figsize=(10,6))` function creates the graph with a size of 10×6.

The `plt.scatter()` function creates the bubble chart where:

- Years of Experience is placed on the X-axis
- Salary After AI is placed on the Y-axis
- Bubble size represents Automation Risk
- Bubble color represents Job Satisfaction

The parameter $\alpha=0.6$ controls the transparency of the bubbles. The `plt.colorbar()` function adds a color scale for Job Satisfaction.

Output:



Interpretation of Bubble chart:

Bubble Chart- A scatterplot where bubble size represents an additional variable.

- It helps understand how experience, salary, automation risk, and satisfaction interact.
- Features: This advanced bubble chart analyzes 4 variables together-X-axis → Years of Experience,Y-axis → Salary After AI,Bubble Size → Automation Risk,Bubble Color → Job Satisfaction
- Larger bubbles represent employees with higher automation risk & Color intensity shows differences in job satisfaction.

- From this, we can identify patterns such as experienced employees earning higher salaries, which groups face higher automation risk, how job satisfaction changes across employee categories.

11. Advanced plot- Violin plot:

Code:

```
# Advanced Plot- Violin Plot

plt.figure(figsize=(10,6))
sns.violinplot(
    x='Education Level',
    y='Job Satisfaction',
    hue='Remote Work',
    data=df,
    split=True
)

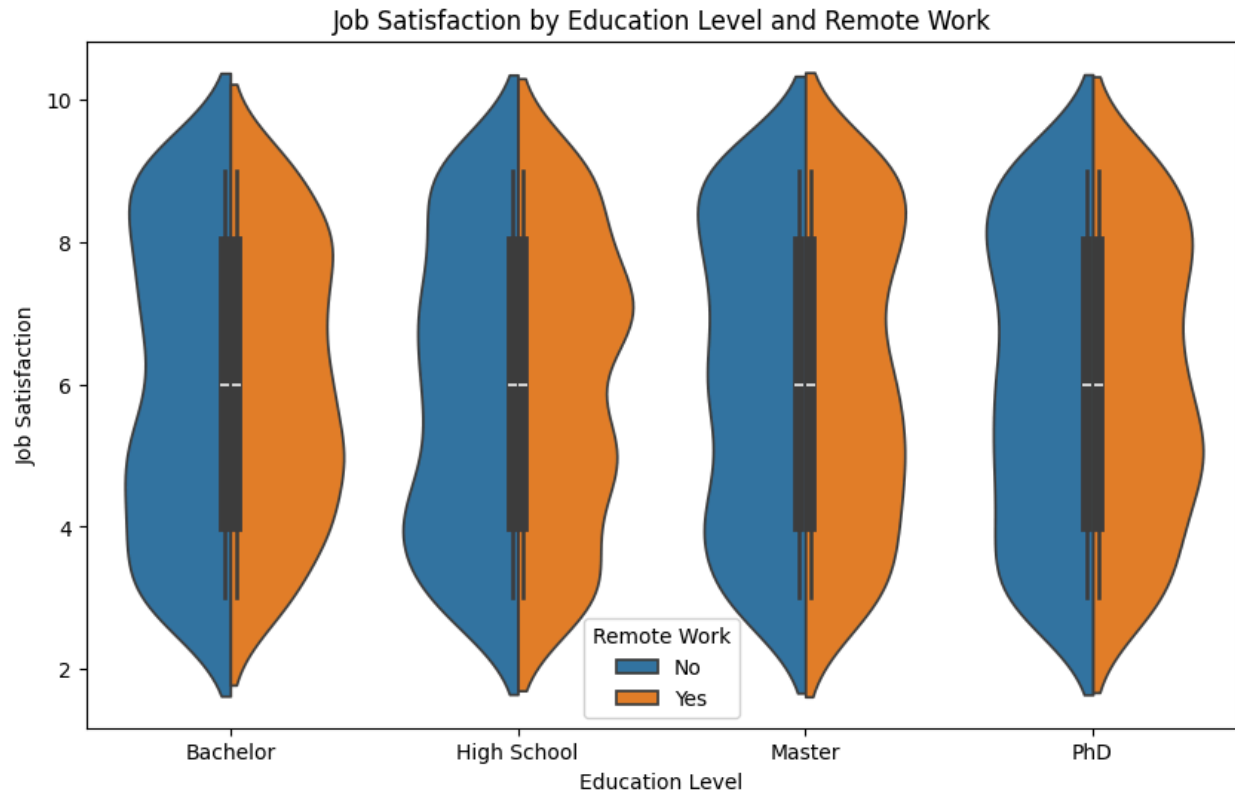
plt.title('Job Satisfaction by Education Level and Remote Work')
plt.xlabel('Education Level')
plt.ylabel('Job Satisfaction')
plt.show()
```

The `sns.violinplot()` function creates the violin plot using:

- `x='Education Level'` to place education levels on the X-axis
- `y='Job Satisfaction'` to place job satisfaction scores on the Y-axis
- `hue='Remote Work'` to separate the data based on remote work categories
- `data=df` to use the dataset for visualization

splits the violin plot into two halves for easy comparison between remote and non-remote employees.

Output:

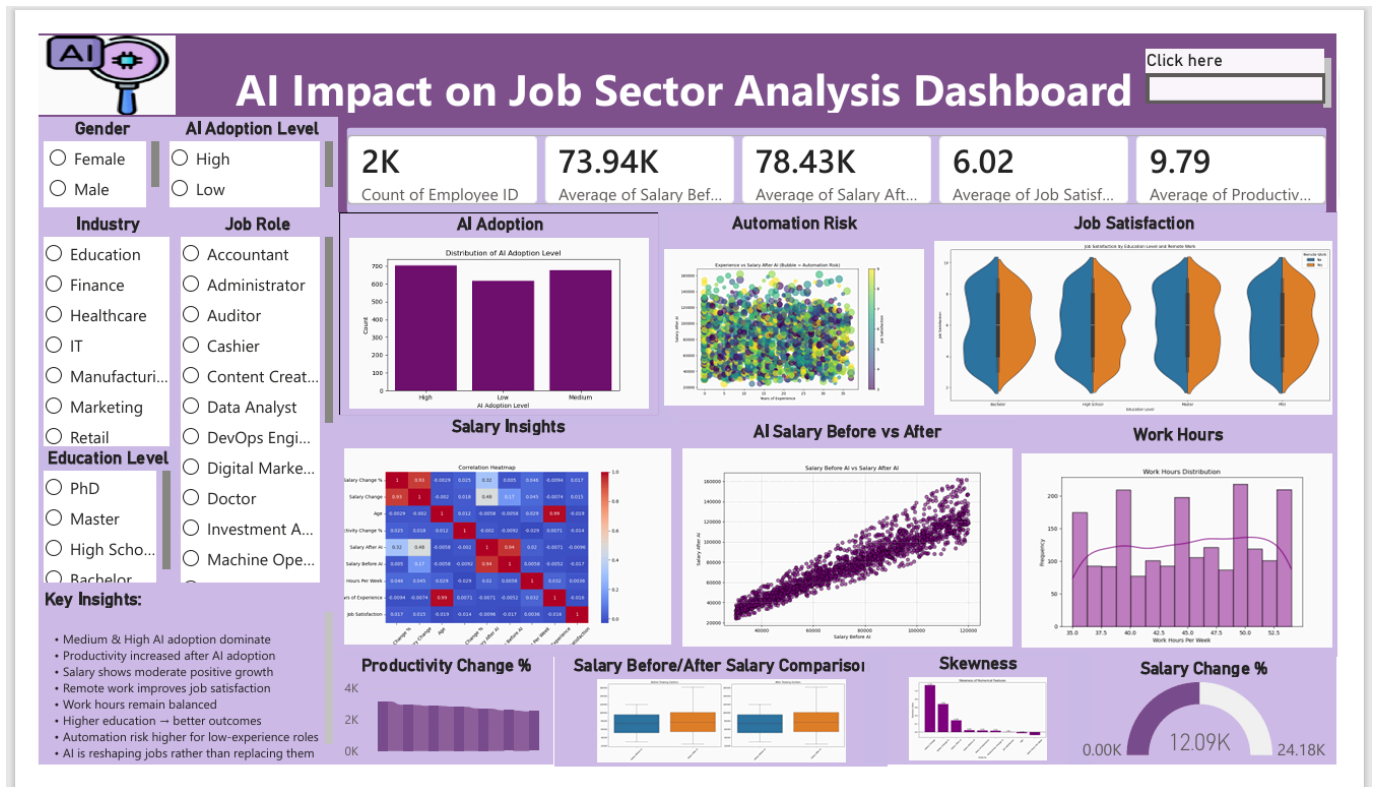


Interpretation of Violin plot:

Violin Plot- Combines boxplot and density plot to show spread and distribution shape.

- This violin plot shows the relationship between Education Level, Job Satisfaction, and Remote Work.
- Features: X-axis → Education Level, Y-axis → Job Satisfaction, Color (Hue) → Remote Work (Yes/No)
- The width of the violin shows where more employees are concentrated & Wider sections indicate more employees having similar job satisfaction scores.
- This helps compare satisfaction levels across education groups and remote work categories & It also helps identify spread, distribution shape, and variation among groups.

Dashboard:



Dashboard Summary:

The **AI Impact on Job Sector Analysis Dashboard** provides a comprehensive overview of how Artificial Intelligence is influencing employee salaries, productivity, job satisfaction, work patterns, and automation risk across different industries and job roles. The dashboard combines multiple visualizations and KPI indicators to support data-driven analysis and decision-making.

At the top of the dashboard, key performance indicators (KPIs) display important summary metrics:

- Total Employee Count: **2K**
- Average Salary Before AI: **73.94K**
- Average Salary After AI: **78.43K**
- Average Job Satisfaction: **6.02**

- Average Productivity Change %: **9.79**

The left-side panel contains slicers for filtering data based on:

- Gender
- AI Adoption Level
- Industry
- Job Role
- Education Level

These filters allow users to interactively explore the dataset and compare insights across different employee categories.

The dashboard includes several visualizations to analyze AI impact:

- **Bar Chart** shows the distribution of AI adoption levels.
- **Bubble Chart** analyzes the relationship between experience, salary after AI, automation risk, and job satisfaction.
- **Violin Plot** compares job satisfaction across education levels and remote work categories.
- **Correlation Heatmap** displays relationships between numerical features such as salary, productivity, and work hours.
- **Scatter Plot** compares Salary Before AI and Salary After AI to visualize salary growth trends.
- **Histogram** displays the distribution of employee work hours.
- **Boxplots** compare salary distributions before and after AI adoption.
- **Gauge Chart** highlights overall salary change percentage.
- **Skewness Chart** shows the distribution symmetry of numerical features.

The dashboard insights indicate that:

- Medium and High AI adoption levels dominate the dataset.
- Productivity generally increased after AI adoption.
- Salaries show moderate positive growth after AI implementation.

- Remote work slightly improves job satisfaction.
- Work hours remain relatively balanced across employees.
- Higher education levels are associated with better salary outcomes.
- Automation risk is higher for low-experience roles.
- AI is transforming and modifying jobs more often than completely replacing them.

Overall, the dashboard provides an interactive and visual representation of AI's impact on the job sector, helping analyze workforce trends, salary changes, employee satisfaction, productivity improvements, and automation effects across industries.

Key Insights:

1. AI Adoption Increased Productivity

Employees working in organizations with Medium and High AI adoption levels showed higher productivity change percentages, indicating that AI positively impacts employee efficiency and work performance.

2. Salary Growth After AI Implementation

The average salary after AI adoption is higher than the salary before AI adoption, showing that AI implementation contributed to salary improvement in many industries and job roles.

3. Jobs Were Modified More Than Replaced

Most employees remained in the “Unchanged” or “Modified” job status categories, while only a smaller percentage of jobs were completely replaced by AI. This indicates that AI is transforming work roles rather than eliminating jobs entirely.

4. Higher Education Leads to Better Salary Outcomes

Employees with higher educational qualifications such as Master's and PhD generally received higher salaries after AI adoption compared to employees with lower education levels.

5. Strong Correlation Between Old and New Salaries

The correlation analysis and scatter plot show a strong positive relationship between Salary Before AI and Salary After AI, meaning employees with higher previous salaries generally continued to earn higher salaries after AI implementation.

Recommendations for Business and Decision-Making:

1. Invest More in AI Adoption and Employee Training

Organizations should continue adopting AI technologies while providing regular upskilling and reskilling programs for employees. This helps employees adapt to changing job requirements and improves productivity.

2. Focus on Job Transformation Instead of Job Replacement

Since most jobs were modified rather than replaced, businesses should redesign work processes to combine human skills with AI support instead of fully automating roles.

3. Encourage Higher Education and Technical Skill Development

Employees with higher education and technical skills showed better salary outcomes. Companies should encourage certifications, advanced training, and continuous learning programs.

4. Monitor Automation Risk in Repetitive Roles

Industries with repetitive tasks should identify high automation-risk positions and prepare employees through skill development and role transition planning.

5. Promote Remote and Flexible Work Policies

Remote work employees showed slightly better job satisfaction levels. Businesses can improve employee satisfaction and retention by supporting flexible work environments.

6. Use AI to Improve Productivity and Decision-Making

Organizations should use AI tools for data analysis, workflow automation, and operational efficiency to increase productivity without negatively impacting employee performance.

7. Perform Regular Workforce and Salary Analysis

Companies should continuously monitor salary trends, productivity changes, and employee satisfaction to evaluate the long-term impact of AI adoption on the workforce.

8. **Balance Automation with Human Involvement**

Businesses should maintain a balance between AI automation and human decision-making to ensure creativity, innovation, and employee engagement are not affected.

Conclusion:

The **AI Impact on Job Sector Analysis** project successfully analyzed how Artificial Intelligence is influencing employee productivity, salaries, job satisfaction, automation risk, and work patterns across different industries. The analysis revealed that AI adoption generally improves productivity and salary growth while transforming job roles rather than completely replacing employees.

The study also showed that employees with higher education levels and technical skills experience better career outcomes after AI adoption. Most organizations are using AI as a supportive technology to enhance efficiency, automate repetitive tasks, and improve decision-making processes.

Through data preprocessing, visualization, statistical analysis, and dashboard development using Python and Power BI, meaningful insights were generated from the dataset. The project demonstrates that AI can positively impact business performance when combined with proper employee training, upskilling, and balanced workforce management.

Overall, the project highlights that AI is reshaping the future of work by creating smarter, more efficient, and technology-driven work environments while still maintaining the importance of human involvement in organizational success.